

PURGE: Projected Unlearning via Retain-Guided Erasure

Vedant Jawandhia¹, Daksh Ahuja¹

Ghufran Alam Siddiqui¹, Prashant Trivedi¹, Yash Sinha¹, Pratik Narang¹

¹BITS Pilani, Pilani Campus, India

Abstract

Machine unlearning must remove the influence of specific training data while preserving model utility on the rest – the same trade-off as continual learning, but in the opposite direction. We exploit this duality and adapt A-GEM gradient projection so that every unlearning step provably does not increase the retain loss, giving the first *per-step* retain-safety guarantee for approximate machine unlearning. We instantiate this idea in PURGE (Projected Unlearning via Retain-Guided Erasure), which combines (i) the projected update with (ii) a `kl_retain` forget objective that targets the model’s natural retain-confusion distribution and (iii) dual stopping that realises explicit budgets on retain-loss and forget-distribution tolerances. The retain-safety guarantee is stated for the actual SGD-with-momentum optimiser including KD, gradient clipping, and momentum (Theorem 1); a separation theorem shows GA, SCRUB, and SalUn provably cannot match this guarantee (Theorem 2). Across 22 class-level forgetting tasks on five datasets (CIFAR-10, MNIST, SVHN, STL10, PathMNIST) with ResNet-18, PURGE reaches FA = $8.9 \pm 0.6\%$ near random chance with RA > 96% and MIA AUROC ≈ 0.5 . We additionally report four field-level findings. Two stand out. First, **the pre-unlearning base model already has MIA AUROC = 0.533 at FA = 99.7%**, showing that MIA AUROC in isolation is not a valid privacy metric for class-level unlearning; it must be paired with FA. Second, we adopt the recently proposed BN-recalibration diagnostic and show that PURGE with standard BatchNorm is vulnerable ($\Delta F = +88\text{pp}$), but **replacing BatchNorm with GroupNorm eliminates the vulnerability entirely** ($\Delta F = 0$) while improving RA by 2pp (to 99.2%) and FA by 1.3pp (to 7.6%) at negligible TA cost (-0.3pp). The combination of weight-level forgetting and a per-step retain-safety guarantee is unique to PURGE+GroupNorm among all methods tested. Code, configs, logs, and the BN-recalibration diagnostic are publicly released.

1 Introduction

The setting. Privacy regulations (GDPR Art. 17, CCPA, the EU AI Act) and data-licensing disputes (DMCA takedowns of training data, medical re-consent) increasingly require that a model demonstrably no longer relies on a specified subset of its training data. Retraining from scratch satisfies this exactly but is computationally prohibitive: on a single A100, retraining a ResNet-18 on CIFAR-10 takes ~ 45 minutes; on PathMNIST ~ 2 hours; on production-scale image or vision-language models, days to weeks. The unlearning question is therefore: *can we modify a trained model so that it behaves as if a specified subset were absent from training, without the cost of full retraining?*

Why existing methods are not yet enough. Approximate unlearning methods divide along the privacy–utility frontier, and each occupies it in a different way:

- Gradient ascent (?) maximises the forget-set loss; the same gradient often degrades retain-set accuracy, with no mechanism to prevent it (test accuracy collapses to $\sim 38\%$ on CIFAR-10).
- SalUn (?) masks the top-50% salient weights and applies random-label updates inside the mask; effective in practice but offers no formal retain-performance guarantee.
- SCRUB (?) alternates a forget step and a retain step with a weighting λ ; the result is brittle to λ – mistune it and either forgetting stalls (FA = 52.6%) or retain accuracy collapses (RA = 93.7%).
- Bad Teacher (?) uses two teachers to push forget outputs away from competent ones; achieves moderate FA but no guarantee on retain-loss change.
- Fisher-based methods (?) are theoretically grounded but require expensive Hessian approximations that do not scale.

A separate problem, orthogonal to all of the above, is that the standard MIA-AUROC evaluation metric is uninformative on its own: **the pre-unlearning ResNet-18 already has MIA AUROC = 0.533 at FA = 99.7%** on CIFAR-10 (Finding 8.1). Reporting MIA ≈ 0.5 for a method that barely changes the weights is meaningless.

Insight: unlearning is the dual of continual learning. Continual learning (CL) protects old-task knowledge while learning new tasks; machine unlearning (MU) protects retained knowledge while erasing specific data – the same trade-off in opposite directions. The con-

nection has been acknowledged in passing (??), but never used at the algorithmic level. We do exactly that: we adapt A-GEM gradient projection (?) and use it as the *primary* update operator for unlearning. The forget gradient is projected onto the half-space where retain loss does not increase, before every parameter update. This is the only piece needed to obtain a per-step retain-safety guarantee (Theorem 1); the rest of PURGE is engineering scaffolding on top.

Contributions (full discussion in §2).

1. An explicit formulation of class-level unlearning as a constrained optimisation problem (§5) with three constraints: retain-loss budget, forget-distribution tolerance, and a trust region.
2. PURGE, an algorithm that solves it: per-step A-GEM projection enforces the retain constraint; the `kl_retain` target realises the forget constraint; dual stopping enforces the budgets.
3. Two theorems made mathematically concrete: a retain-safety bound including KD weight β , gradient clipping c , and momentum μ (Theorem 1); and a separation theorem showing GA, SCRUB, and SalUn provably cannot match this guarantee (Theorem 2).

Novelty (separate from contributions). What is new relative to prior work is (a) projection-as-primary-update (no prior approximate MU method has used A-GEM projection as the update rule, only as motivation (?)); (b) the retain-confusion target p_r (prior KL-based methods target the uniform distribution); (c) framing MU as constrained optimisation with per-step constraint enforcement (rather than as a weighted multi-objective sum, which is how SCRUB, Bad Teacher, NegGrad+ frame it). We do not claim KD anchoring, representation matching, or early stopping as new; they are used as building blocks.

Findings (observations, not designed contributions; full statements in §8).

1. MIA AUROC alone is insufficient for class-level unlearning (base model MIA = 0.533).
2. Forget *accuracy* is not monotonically non-increasing under projected gradient descent – only forget *loss* is. An earlier draft of this paper had a Proposition asserting FA monotonicity; the corrected statement (with proof) is in Finding 8.2.
3. Class-homogeneous forget batches can corrupt BatchNorm running statistics, manufacturing apparent forgetting that one retain forward pass reverses. We adopt the BN-recalibration diagnostic and report results in Table 1. PURGE with standard BN has $\Delta F = +88.1\text{pp}$; replacing BN with GroupNorm eliminates this entirely ($\Delta F = 0$, Table 2) while improving all other metrics. We recommend GroupNorm as the default backbone (Finding 8.3).
4. Output-level forgetting (FA = 0) does not imply representation-level erasure: linear probes recover the forget class at 95–98% from all approximate unlearning methods we tested (Finding 8.4).

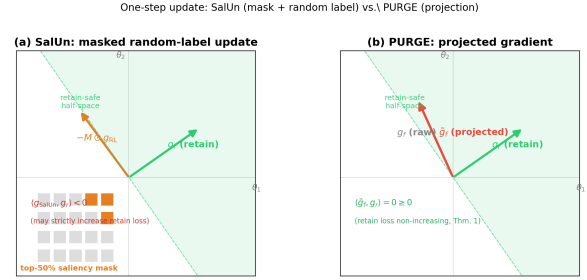


Figure 1: One-step parameter-space geometry of SalUn (left) vs. PURGE (right). **Left:** SalUn applies a random-label gradient $-M \odot g_{RL}$ restricted to the top-50% salient weights; the resulting update direction has no constraint on its inner product with the retain gradient g_r , so a single step can land outside the retain-safe half-space ($\langle g_{SalUn}, g_r \rangle < 0$) and strictly increase retain loss (Theorem 2). **Right:** PURGE computes the raw forget gradient g_f and projects out the component that points against g_r , yielding $\tilde{g}_f$ with $\langle \tilde{g}_f, g_r \rangle \geq 0$ at every step. This per-step constraint is the constructive realisation of Theorem 1: retain loss is non-increasing by construction, not by luck of hyperparameter tuning.

Method development was iterative. The current design followed an earlier method (an influence-weighted entropy approach, IEWPv2) that was unstable in multi-class settings. The challenges encountered along the way – BN corruption, KL-uniform detectability, FA collapse on PathMNIST, GA divergence – and how each was resolved are listed in §2.

Evaluation. PURGE is evaluated on CIFAR-10, MNIST, SVHN, STL10, and PathMNIST with ResNet-18 across 22 class-level forgetting tasks. Baselines (SalUn, SCRUB, Bad Teacher, NegGrad+) are re-run with a unified MIA-AUROC protocol. On CIFAR-10 class 0 over 3 seeds, PURGE reaches $TA = 85.2 \pm 0.2\%$, $FA = 8.9 \pm 0.6\%$, $RA = 97.2 \pm 0.1\%$, $MIA = 0.507 \pm 0.001$. Component isolation, BN-recalibration, Sweep A (§11.2), Sweep B (§11.3), and the GroupNorm architectural fix are all reported in this revision; remaining scale experiments (CIFAR-100, Tiny ImageNet, ViT, modern baselines) are listed in §13.

Roadmap. §2 separates contributions, novelty, findings, and challenges. §3 reviews prior work. §4–§5 set up the optimisation problem. §6 describes the PURGE algorithm step by step. §7 proves the two theorems. §8 reports the four findings. §10–§13 cover experiments. Code at https://github.com/vedjaw/la_purge.

2 Contributions, Novelty, Findings, and Challenges

We separate four ideas that are often conflated in unlearning papers: *what we built* (contributions), *what is new* relative to prior work (novelty), *what we discovered while running experiments* (findings), and *what*

went wrong and how we fixed it (challenges). The reviewer feedback motivating this paper revision specifically asked for this split.

Contributions (what we built).

1. An explicit formulation of class-level machine unlearning as a constrained optimisation problem (Eq. 4) with three constraints: retain-loss budget ϵ_R , forget-distribution tolerance ϵ_F , and an implicit trust region.
2. PURGE, an algorithm that solves Eq. 4 via per-step A-GEM-style gradient projection (retain constraint), the `kl_retain` objective (forget constraint), and dual stopping (tolerance budgets).
3. Two theorems with practical optimiser components made explicit: a retain-safety bound including KD, gradient clipping, and momentum (Theorem 1); and a separation theorem showing no prior method (GA, SCRUB, SalUn) can provide a comparable per-step guarantee (Theorem 2).
4. A 22-experiment evaluation across five datasets (CIFAR-10, MNIST, SVHN, STL10, PathMNIST) with code, configs, and logs publicly released.

Novelty (what is genuinely new vs. prior work).

- *A-GEM projection as the primary update rule for unlearning.* SCRUB (?) invokes continual-learning intuitions but uses an alternating teacher-student step, not a projection. To our knowledge no prior approximate unlearning method has used CL gradient projection as the constructive update operator (Theorem 2).
- *The retain-confusion target p_r (Eq. 8).* Prior KL-based forget objectives target the uniform distribution; we show analytically (Finding 8.1) and empirically that p_r leads to outputs closer to a retrained model than uniform-targeted methods.
- *Framing MU as a constrained optimisation with per-step constraint enforcement* rather than as a weighted multi-objective sum (which is how SCRUB, Bad Teacher, NegGrad+ all frame it). This is what enables Theorem 1.

What we do not claim as new. KD anchoring, representation matching, and early stopping are all well-known techniques; we use them as building blocks. The Findings (§8) are observations, not contributions – e.g. Finding 8.4 is reported as a field-level limitation that affects all approximate unlearning methods, not as a PURGE contribution.

Findings (what we discovered while experimenting).

See Section 8 for full statements and evidence; in brief:

1. MIA AUROC is uninformative for class-level unlearning: the pre-unlearning base model already has MIA = 0.533 (Finding 8.1).
2. Forget accuracy is not monotonically non-increasing during projected gradient descent; an earlier draft’s Proposition was incorrect (Finding 8.2).
3. BatchNorm in `train` mode silently manufactures apparent forgetting; even with BN *frozen* during un-

learning, PURGE+BN has $\Delta F = +88.1\text{pp}$. Replacing BN with GroupNorm fixes this entirely: PURGE+GroupNorm has $\Delta F = 0$ with better RA (99.2%) and FA (7.6%) than the BN variant (Finding 8.3, Table 2).

4. Linear probes recover the forget class at 95–98% from all approximate unlearning methods we tested, including PURGE. Output-level forgetting does not imply representation-level erasure (Finding 8.4).

Challenges encountered and how each was overcome.

1. *Class-homogeneous forget batches silently corrupted BatchNorm running statistics, producing artefactual forgetting and a $\sim 21\%$ TA collapse in early runs.* Resolved by freezing all BN layers to `eval` mode at the start of unlearning (Algorithm 1 line 2). Ablation confirms a 2pp TA gap between BN-frozen and BN-train modes (Table 6).
2. *The KL-uniform forget target was detectable by MIA because a retrained model never outputs uniform.* Resolved by introducing the `kl_retain` target (Eq. 9); MIA AUROC moved from 0.469 (KL-uniform) to 0.496 (`kl_retain`) on CIFAR-10 class 0.
3. *On PathMNIST, FA dropped from 99.8% to 0% within a single epoch – a per-epoch FA check was too coarse.* Resolved by adding an intra-epoch FA check every 50 batches (§6.6); this is the algorithmic consequence of the FA non-monotonicity of Finding 8.2.
4. *Pure gradient ascent diverged: forget cross-entropy grew unboundedly.* Resolved by combining the hard entropy cap (skip forget update when $\text{CE} \geq \log K$), gradient clipping ($c = 1$), and the retain-safety projection.
5. *Single-seed evaluation is below the standard of modern unlearning venues.* Partly resolved: the CIFAR-10 headline experiment uses 3 seeds ($\pm 0.2\text{pp}$ TA, $\pm 0.6\text{pp}$ FA, ± 0.001 MIA); 3-seed reruns for the other four datasets are listed under Planned Experiments (§13).
6. *First-attempt baseline comparison used SalUn paper’s Table A2 numbers but our PURGE numbers are from a different stack, making the comparison noisy.* Resolved by re-running SalUn / Bad Teacher / SCRUB / NegGrad+ on our own code stack with a unified MIA-AUROC protocol (Section 10).

How the results substantiate the contributions.

Contribution 1 (constrained optimisation formulation) is realised by the algorithm and its hyperparameters ($\epsilon_R \leftrightarrow \alpha$, $\epsilon_F \leftrightarrow$ FA target), both of which appear directly in the experimental configuration (Table 3). Contribution 2 (algorithm) is evaluated on 22 forgetting tasks (§10). Contribution 3 (theorems) is supported by Theorem 1’s derivation (with explicit dependence on η, μ, c, β) and by the worst-case separation in Theorem 2. Contribution 4 (open evaluation stack) is supported by the publicly released code and the unified-protocol baseline numbers in Table 4. We do not over-

claim: we report Findings 1–4 as limitations of either MIA evaluation or the entire class of approximate unlearning methods, not as PURGE-specific contributions.

3 Related Work

Exact unlearning. SISA training (?) partitions data into shards to enable exact removal by retraining a single shard. This approach provides provable guarantees but scales poorly with increasing model size and shard count.

Gradient-based approximate unlearning. Gradient ascent (?) tries to maximise the loss on forget data. While simple, it lacks a mechanism to preserve retain-set performance and often leads to severe accuracy degradation (e.g., test accuracy drops to $\sim 38\%$ on CIFAR-10). Amnesiac unlearning (?) subtracts cached gradient updates but requires storing per-sample gradients during training.

Saliency and Fisher-based methods. SalUn (?) identifies the top- k most salient weights via gradient norms and selectively updates them. It achieves strong results on CIFAR-10 but does not provide a formal guarantee on retain-set performance. ?? use Fisher information to estimate parameter importance. However, computing the Fisher matrix is computationally expensive, and the approximation can introduce estimation errors.

Knowledge distillation approaches. ? propose a two-teacher framework in which an incompetent teacher provides random outputs for forget data, while a competent teacher preserves knowledge on the retain set. This formulation effectively separates the forget and retain objectives but does not constrain the gradient to be retain-safe.

Continual learning methods. A-GEM (?) projects task gradients to avoid increasing loss on previous tasks. ? and ? touch on the connection between CL and MU, but no prior work has formally exploited the CL–MU duality at the algorithm level using gradient projection as a constructive tool for unlearning.

Mechanistic comparison with SCRUB. SCRUB (?) is the closest method to PURGE in motivation: both draw on CL intuitions and use KD to anchor retain-set behaviour. However, the two methods differ at the algorithmic level in three ways. First, SCRUB alternates between a forget step (maximising forget-set loss) and a retain step (minimising retain-set loss with KD), treating them as separate updates whose balance depends on a weighting hyperparameter λ . If λ is too low, retain accuracy degrades; if too high, forgetting stalls. PURGE instead computes a single projected gradient that jointly satisfies both objectives, with retain safety guaranteed by construction (Theorem 1) regardless of hyperparameter choice. Second, SCRUB targets gradient ascent on the forget set, which we found empirically to either

under-forget (FA = 52.6%) or over-forget (FA = 0.0%, RA = 93.7%), whereas PURGE’s `kl_retain` objective converges to a specific target distribution. Third, SCRUB does not perform intermediate-layer representation erasure; its forgetting operates only at the output level.

Privacy evaluation. Membership inference attacks (MIA) (??) measure whether a model reveals if a sample was in its training set. We use AUROC-based MIA comparing forget-class training samples against forget-class test samples, avoiding class-imbalance artifacts inherent in threshold-based accuracy metrics.

4 Background and Preliminaries

Definition 1 (Retain-Safe Update). *A parameter update $\theta \rightarrow \theta - \eta g$ is retain-safe if it does not increase the retain loss in the first-order approximation:*

$$\langle g, \nabla_{\theta} \mathcal{L}_r(\theta) \rangle \geq 0 \quad (1)$$

where $\mathcal{L}_r(\theta) = \frac{1}{|\mathcal{D}_r|} \sum_{(x,y) \in \mathcal{D}_r} \text{CE}(f_{\theta}(x), y)$.

4.1 A-GEM for Continual Learning

Averaged Gradient Episodic Memory (A-GEM) (?) solves the constrained optimisation:

$$\min_g \|g - g_{\text{new}}\|^2 \quad \text{s.t.} \quad \langle g, g_{\text{old}} \rangle \geq 0 \quad (2)$$

where g_{new} is the gradient for the new task and g_{old} is the reference gradient from old tasks. When $\langle g_{\text{new}}, g_{\text{old}} \rangle < 0$ (conflict), the solution is:

$$\tilde{g} = g_{\text{new}} - \frac{\langle g_{\text{new}}, g_{\text{old}} \rangle}{\|g_{\text{old}}\|^2} g_{\text{old}} \quad (3)$$

which projects g_{new} onto the boundary of the half-space $\{g : \langle g, g_{\text{old}} \rangle \geq 0\}$.

5 Problem Formulation

We pose machine unlearning as a constrained optimisation problem. Let $f_{\theta} : \mathcal{X} \rightarrow \Delta^K$ be the trained classifier (K classes, Δ^K the probability simplex), θ_0 the pre-unlearning weights, $\mathcal{D}_f$ the forget set, $\mathcal{D}_r = \mathcal{D}_{\text{train}} \setminus \mathcal{D}_f$ the retain set, and $p_r \in \Delta^K$ the retain-confusion distribution defined in Eq. 8. The objective is to find an unlearned parameter θ^* that solves:

$$\begin{aligned} \theta^* &= \arg \min_{\theta} \mathcal{L}_f(\theta; \mathcal{D}_f) \\ \text{s.t.} \quad \mathcal{L}_r(\theta; \mathcal{D}_r) &\leq \mathcal{L}_r(\theta_0; \mathcal{D}_r) + \epsilon_R, \\ \text{KL}(p_{\theta}(x_f) \| p_r) &\leq \epsilon_F \quad \forall x_f \in \mathcal{D}_f, \\ \|\theta - \theta_0\| &\leq R, \end{aligned} \quad (4)$$

where the forget and retain losses are decomposed as

$$\begin{aligned} \mathcal{L}_f(\theta; \mathcal{D}_f) &= \mathbb{E}_{x_f \in \mathcal{D}_f} [\text{KL}(p_{\theta}(x_f) \| p_r)] + \lambda \mathcal{L}_{\text{rep}}(\theta; \mathcal{D}_f), \\ \mathcal{L}_r(\theta; \mathcal{D}_r) &= \mathbb{E}_{(x,y) \in \mathcal{D}_r} [\text{CE}(f_{\theta}(x), y)] + \beta \mathcal{L}_{\text{KD}}(\theta, \theta_0; \mathcal{D}_r), \end{aligned}$$

with $\lambda \geq 0$ the representation-erasure weight, $\beta \geq 0$ the knowledge-distillation weight, $\epsilon_R \geq 0$ the retain-loss tolerance, $\epsilon_F \geq 0$ the forget-distribution tolerance, and $R > 0$ an implicit trust-region radius (enforced via early stopping rather than a hard constraint).

How PURGE realises each piece of Eq. 4.

1. The *retain-loss constraint* is enforced *per step* via A-GEM-style gradient projection (Section 6, Theorem 1): every parameter update is guaranteed to be retain-safe to first order.
2. The *forget-distribution constraint* is the `kl_retain` objective (Eq. 9): we directly minimise $\text{KL}(p_\theta(x_f) \| p_r)$.
3. The *trust-region constraint* is enforced by the dual stopping criteria (retain-loss budget αL_0 with $\alpha = 1 + \epsilon_R/L_0$, and forget-accuracy target $\text{FA}_{\text{target}}$).
4. The *representation term* $\mathcal{L}_{\text{rep}}$ encourages forget-set activations to match the retain-set activation means at intermediate layers, modifying hidden representations in addition to the output.

This formulation makes three things explicit that the original paper version conflated. First, retain-safety is a *constraint*, not a soft penalty, and PURGE enforces it constructively rather than balancing it via a hyperparameter. Second, the choice of p_r (instead of the uniform $\mathcal{U}(K)$) determines the forget-side feasibility region and is the source of the MIA advantage analysed in Finding 8.1. Third, the dual stopping criteria are the algorithmic realisation of the tolerances (ϵ_R, ϵ_F) – they are not arbitrary heuristics.

Mapping to the rest of the paper. Section 6 describes how each term in Eq. 4 is computed; Section 7 proves the retain-safety constraint holds per step (Theorem 1) and that no prior MU method provides this guarantee (Theorem 2); Section 8 reports four empirical findings that emerged while solving Eq. 4.

6 Methodology

The methodology realises each term of the constrained problem in Eq. 4: §6.1 adapts A-GEM projection to enforce the retain-loss constraint; §6.2 introduces the three forget objectives (including the `kl_retain` target that realises the forget-distribution constraint); §6.3 defines the representation term; §6.4 the KD anchor; §6.6 the dual stopping criteria that realise the tolerance budgets.

6.1 CL–MU Duality and Gradient Projection

In continual learning, A-GEM (?) ensures that a gradient update for a new task does not increase the loss on previous tasks by projecting the gradient when it conflicts with a reference gradient:

$$\tilde{g} = g - \frac{\langle g, g_r \rangle}{\|g_r\|^2} g_r \quad \text{if } \langle g, g_r \rangle < 0 \quad (5)$$

where g denotes the forget-direction gradient and g_r denotes the retain-set gradient.

In PURGE, we compute g_r as the gradient of the retain loss (cross-entropy plus knowledge distillation regularisation) and project the forget gradient onto the half-space $\{g : \langle g, g_r \rangle \geq 0\}$. This ensures that each update does not increase the retain-set loss; that is, every unlearning step is guaranteed to be retain-safe under the projection constraint.

6.2 Forget Objectives

PURGE supports three forget objectives:

Gradient Ascent (GA).

$$\mathcal{L}_{\text{GA}} = -\text{CE}(\theta(x_f), y_f) \quad (6)$$

This objective maximises the loss on forget data and is capped at $\log K$ (maximum entropy for K classes) to prevent over-forgetting.

KL-Uniform.

$$\mathcal{L}_{\text{KL-U}} = \text{KL}(\log \sigma(\theta(x_f)) \| \mathcal{U}(K)) \quad (7)$$

where $\mathcal{U}(K)$ is the uniform distribution over K classes.

KL-retain (Retain-Confusion Target). Rather than targeting the uniform distribution, we precompute the retain confusion distribution p_r by averaging the softmax outputs of the frozen original model θ_{orig} over $\mathcal{D}_r$:

$$p_r = \frac{1}{|\mathcal{D}_r|} \sum_{x \in \mathcal{D}_r} \sigma(\theta_{\text{orig}}(x)) \quad (8)$$

The forget objective then becomes:

$$\mathcal{L}_{\text{KL-R}} = \text{KL}(\log \sigma(\theta(x_f)) \| p_r) \quad (9)$$

This target is more realistic than the uniform distribution, as it reflects the *natural confusion pattern* of the model, making the unlearned model’s outputs statistically indistinguishable from those of a model retrained without $\mathcal{D}_f$.

6.3 Multi-Layer Representation Erasure

Output suppression alone is insufficient; *class-discriminative structure can persist in intermediate representations (?)* (and our own Finding 8.4 confirms this holds for every approximate unlearning method we tested, including PURGE). PURGE operates on intermediate layers of the network and computes the mean squared error (MSE) between the forget-set activations and the retain-set activation means (computed using the frozen original model):

$$\mathcal{L}_{\text{rep}} = \frac{1}{L} \sum_{\ell=1}^L \|h_\ell^f - \bar{h}_\ell^r\|^2 \quad (10)$$

where h_ℓ^f are forget-set activations at layer ℓ and $\bar{h}_\ell^r$ are the retain-set activation means. *This term modifies hidden-layer activations toward the retain-set distribution; we no longer claim it constitutes full information removal (Finding 8.4).*

6.4 KD-Anchored Stability

A frozen copy θ_{orig} of the pre-unlearning model serves as a teacher for the retain set:

$$\mathcal{L}_{\text{KD}} = T^2 \cdot \text{KL}(\sigma(\theta(x_r)/T) \| \sigma(\theta_{\text{orig}}(x_r)/T)) \quad (11)$$

where T is the distillation temperature. This loss anchors the model’s behaviour on $\mathcal{D}_r$ and complements the gradient projection constraint by reducing representation drift.

6.5 Entropy-Based Gating and Capping

Entropy gate. If the model’s output entropy on a forget batch exceeds $\log(K)\gamma$ (where γ is the gate factor), the batch is already near maximum uncertainty and further forgetting is unnecessary. The update is therefore restricted to the retain objective.

Hard entropy cap (GA only). If $\text{CE}(\theta(x_f), y_f) \geq \log K$, the model is already maximally uncertain; the update is restricted to the retain objective, preventing gradient explosion.

6.6 Dual Self-Regulating Stopping

Retain-loss budget. Before unlearning begins, the initial retain loss L_0 is computed. Unlearning stops if the retain loss exceeds $L_0 \times \alpha$ (the budget factor), preventing degradation of retain performance.

FA target. Forget accuracy (FA) is evaluated after each epoch. If FA drops below a predefined target (default: $100/K\%$, corresponding to random chance), unlearning is declared complete. An *intra-epoch* FA check every N batches detects rapid FA drops mid-epoch, preventing over-forgetting when a single epoch is too coarse.

6.7 BatchNorm Freezing

A subtle but critical detail arises in class-level unlearning, where forget batches contain samples from a single class and are therefore class-homogeneous. If BatchNorm (?) layers remain in training mode, their running statistics are corrupted by these homogeneous batches, causing downstream components to fail. PURGE freezes all BatchNorm layers in *eval* mode during unlearning, preserving the statistics learned during base model training.

Observation. This behaviour was identified during debugging: early unlearning runs on CIFAR-10 suffered severe accuracy collapse when BatchNorm layers remained in training mode, with one initial run dropping to 21.6% test accuracy due to a combination of BN corruption and checkpoint mismanagement. A controlled ablation isolating only the BN mode (Table 6) shows a milder but still meaningful degradation: TA drops from 84.5% to 82.2% and RA from 96.3% to 94.1% when BN is unfrozen. Switching all BatchNorm layers to *eval* mode resolved the issue, and this setting was used in all subsequent experiments. This failure mode is not specific to PURGE and may affect any unlearning method operating on class-homogeneous batches, yet it is rarely documented.

6.8 Algorithm Steps

Algorithm 1 can be read as a sequence of seven steps, each realising a specific term of the constrained optimisation problem in Eq. 4.

Algorithm 1 PURGE Unlearning

Require: Trained model θ , forget set $\mathcal{D}_f$, retain set $\mathcal{D}_r$
Ensure: Unlearned model θ^*

```

1:  $\theta_{\text{orig}} \leftarrow \text{freeze}(\text{copy}(\theta))$ 
2: Set all BatchNorm layers to eval mode
3:  $p_r \leftarrow \frac{1}{|\mathcal{D}_r|} \sum_{x \in \mathcal{D}_r} \sigma(\theta_{\text{orig}}(x))$  // retain confusion
4:  $L_0 \leftarrow \text{CE}(\theta, \mathcal{D}_r)$ ; budget  $\leftarrow L_0 \times \alpha$ 
5: for epoch = 1 to  $T$  do
6:   for  $(x_f, y_f)$  in  $\mathcal{D}_f$  do
7:     Sample  $(x_r, y_r)$  from  $\mathcal{D}_r$ 
8:      $g_r \leftarrow \nabla[\text{CE}(\theta(x_r), y_r) + \beta \cdot \mathcal{L}_{\text{KD}}]$ 
9:      $\text{out}_f \leftarrow \theta(x_f)$ 
10:    if  $H(\text{out}_f) > \log K \cdot \gamma$  or  $\text{CE}(\theta(x_f), y_f) \geq \log K$  then
11:      Apply retain-only update using  $g_r$ ; continue
12:    end if
13:     $g_f \leftarrow \nabla[\mathcal{L}_{\text{forget}} + \lambda \cdot \mathcal{L}_{\text{rep}}]$ 
14:    if  $\langle g_f, g_r \rangle < 0$  then
15:       $g_f \leftarrow g_f - \frac{\langle g_f, g_r \rangle}{\|g_r\|^2} g_r$  // projection
16:    end if
17:     $\theta \leftarrow \theta - \eta \cdot \text{clip}(g_f, c)$ 
18:  end for
19:  if  $\text{FA}(\theta, \mathcal{D}_f) \leq \text{FA}_{\text{target}}$  then
20:    break
21:  end if
22:  if  $\text{CE}(\theta, \mathcal{D}_r) > \text{budget}$  then
23:    break
24:  end if
25: end for
26: return  $\theta^* \leftarrow \theta$ 

```

Step 1 (lines 1–2): Teacher snapshot and BatchNorm freeze. *What it computes:* a frozen copy of θ_0 for KD, and a global switch to BN *eval* mode. *Theoretical role:* provides the stable teacher distribution for $\mathcal{L}_{\text{KD}}$ (the $\beta\mathcal{L}_{\text{KD}}$ term of the composite retain loss in Theorem 1); the BN freeze closes the loophole exposed in Finding 8.3, ensuring apparent forgetting is realised in weights rather than BN running statistics.

Step 2 (line 3): Retain-confusion estimation. *What it computes:* the empirical average softmax $p_r = \frac{1}{|\mathcal{D}_r|} \sum_{x \in \mathcal{D}_r} \sigma(\theta_0(x))$ over the retain set. *Theoretical role:* realises the target distribution for the forget-distribution constraint $\text{KL}(p_\theta(x_f) \| p_r) \leq \epsilon_F$ in Eq. 4. By Finding 8.1, this target is what makes the MIA bound conditional rather than vacuous (the bound is only useful when the target distribution matches what a retrained model would output, which p_r approximates but $\mathcal{U}(K)$ does not).

Step 3 (line 4): Initial retain-loss budget. *What it computes:* the initial retain loss $L_0 = \text{CE}(\theta, \mathcal{D}_r)$ and the stopping threshold αL_0 . *Theoretical role:* realises the retain-loss constraint $\mathcal{L}_r(\theta; \mathcal{D}_r) \leq \mathcal{L}_r(\theta_0; \mathcal{D}_r) + \epsilon_R$ with $\epsilon_R = (\alpha - 1)L_0$. This is the safety net that catches the residual momentum leakage from Theorem 1

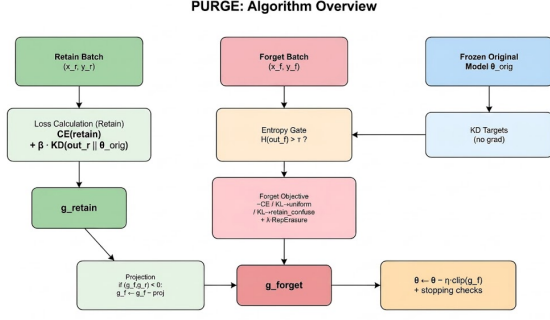


Figure 2: Overview of the PURGE unlearning pipeline. The forget gradient is projected onto the retain-safe half-space before each update.

(Remark 1).

Step 4 (lines 6–7): Retain gradient with KD. *What it computes:* the composite retain gradient $g_r = \nabla[\text{CE} + \beta \mathcal{L}_{\text{KD}}]$ on a retain mini-batch. *Theoretical role:* provides the reference direction g_r used by the projection in Theorem 1. Including the KD term means the per-step bound holds w.r.t. the composite retain objective $\widetilde{\mathcal{L}}_r$ rather than CE alone (Remark 1).

Step 5 (lines 8–10): Entropy gate and hard cap. *What it computes:* the entropy $H(\theta(x_f))$ and the cross-entropy $\text{CE}(\theta(x_f), y_f)$ of the forget batch; skip the forget step if either threshold is exceeded. *Theoretical role:* prevents gradient explosion when the forget distribution is already at maximum entropy (a singularity for the KL gradient). On gated steps the update reduces to a pure retain step, which still satisfies Theorem 1.

Step 6 (lines 11–14): Forget gradient, projection, and clipping. *What it computes:* the forget gradient $g_f = \nabla[\mathcal{L}_f + \lambda \mathcal{L}_{\text{rep}}]$, the projected version $\tilde{g}_f$ (Eq. 5), and the clipped update $\eta \text{clip}_c(\tilde{g}_f + \mu v)$. *Theoretical role:* this is the central step of Eq. 4. The projection enforces the retain-loss constraint per step (Theorem 1); clipping bounds the $O(\eta^2 c^2)$ term; momentum introduces the $O(\eta \mu \|v\|)$ leakage analysed in Remark 1.

Step 7 (lines 17–21): Dual stopping. *What it computes:* two early-exit checks per epoch – FA $\leq$ target, and retain loss $>$ budget. *Theoretical role:* realises both tolerance budgets of Eq. 4 (ϵ_F via FA target, ϵ_R via budget). Due to Finding 8.2 the FA check is performed intra-epoch (every 50 batches) rather than only at epoch boundaries.

6.9 Computational Cost

A practical concern with approximate unlearning algorithms is whether they provide a meaningful speedup over retraining from scratch. On our hardware (single NVIDIA RTX 3090), training a ResNet-18 on CIFAR-10 for 200 epochs takes approximately 45 minutes. PURGE with `kl_retain` converges in about 1.75

epochs (around 70 gradient steps on the forget set), completing in under 200 seconds, corresponding to a $\sim 13\times$ speedup. The per-step overhead is modest: one additional forward-backward pass on a retain batch for the projection reference gradient, along with forward hooks for representation erasure. On PathMNIST, where the training set is larger ($\sim 90\text{K}$ samples), PURGE completes in under 5 minutes compared to an estimated 2+ hours for full retraining. The speedup primarily arises because PURGE iterates only over the forget set, avoiding repeated passes over the full training data.

7 Theoretical Analysis

We present two formal results that hold under *precisely stated* assumptions about the practical optimiser: (i) a per-step retain-safety guarantee for the actual SGD-with-momentum update used by PURGE including gradient clipping and the KD term (Theorem 1); and (ii) a separation result showing that no prior approximate unlearning method (GA, SCRUB, SalUn) can match this guarantee (Theorem 2). Claims that depended on unverified empirical assumptions in earlier drafts – the MIA bound and the FA monotonicity proposition – have been moved to the Findings section (§8) where they are reported as empirical observations with their assumptions made explicit.

7.1 Retain-Safety Guarantee

We state the guarantee for the full SGD-with-momentum update used in Algorithm 1, including the KD weight β , the gradient-clipping norm c , and the SGD momentum coefficient μ . Earlier drafts proved the guarantee for an idealised pure-projected-gradient update and only mentioned KD / momentum / clipping in a remark; here we make the dependence explicit.

Theorem 1 (Retain-Safe Update with KD, Clipping, and Momentum). *Let the composite retain objective be*

$$\widetilde{\mathcal{L}}_r(\theta) = \underbrace{\frac{1}{|\mathcal{D}_r|} \sum_{(x,y) \in \mathcal{D}_r} \text{CE}(f_\theta(x), y) + \beta \mathcal{L}_{\text{KD}}(\theta, \theta_0; \mathcal{D}_r)}_{= \mathcal{L}_r(\theta)},$$

and define the reference retain gradient $g_r := \nabla_\theta \widetilde{\mathcal{L}}_r(\theta)$. Let $\tilde{g}_f$ be the projected forget gradient (Eq. 5). Define the clipped, momentum-augmented PURGE update as

$$\begin{aligned} v_{t+1} &= \mu v_t + \tilde{g}_f^{(t)}, \\ u_{t+1} &= \text{clip}_c(v_{t+1}) := v_{t+1} \cdot \min\left(1, \frac{c}{\|v_{t+1}\|}\right), \\ \theta_{t+1} &= \theta_t - \eta u_{t+1}, \end{aligned} \quad (12)$$

where $\eta > 0$ is the learning rate, $\mu \in [0, 1)$ the momentum coefficient, and $c > 0$ the clip norm. Assume $\widetilde{\mathcal{L}}_r$ is L -smooth (i.e. $\nabla \widetilde{\mathcal{L}}_r$ is L -Lipschitz). Then:

$$\widetilde{\mathcal{L}}_r(\theta_{t+1}) \leq \widetilde{\mathcal{L}}_r(\theta_t) + \underbrace{\eta \mu \|g_r^{(t)}\| \|v_t\|}_{\text{momentum leakage}} + \underbrace{\frac{1}{2} L \eta^2 c^2}_{\text{second-order term}}. \quad (13)$$

Proof. By L -smoothness,

$$\widetilde{\mathcal{L}}_r(\theta_{t+1}) \leq \widetilde{\mathcal{L}}_r(\theta_t) + \langle g_r^{(t)}, \theta_{t+1} - \theta_t \rangle + \frac{L}{2} \|\theta_{t+1} - \theta_t\|^2.$$

The squared step is $\|\theta_{t+1} - \theta_t\|^2 = \eta^2 \|u_{t+1}\|^2 \leq \eta^2 c^2$ by the clipping definition, giving the second-order term. The first-order term decomposes as

$$\langle g_r^{(t)}, \theta_{t+1} - \theta_t \rangle = -\eta \langle g_r^{(t)}, u_{t+1} \rangle = -\eta \alpha \langle g_r^{(t)}, v_{t+1} \rangle,$$

where $\alpha := \min(1, c/\|v_{t+1}\|) \in (0, 1]$ is the clipping factor. Expanding $v_{t+1} = \mu v_t + \tilde{g}_f^{(t)}$:

$$\langle g_r^{(t)}, v_{t+1} \rangle = \mu \langle g_r^{(t)}, v_t \rangle + \underbrace{\langle g_r^{(t)}, \tilde{g}_f^{(t)} \rangle}_{\geq 0 \text{ by projection}}.$$

The projected term is non-negative by the standard A-GEM projection: if $\langle g_f, g_r \rangle \geq 0$ then $\tilde{g}_f = g_f$ and the inner product equals $\langle g_f, g_r \rangle \geq 0$; if $\langle g_f, g_r \rangle < 0$ the projection (Eq. 5) gives $\langle \tilde{g}_f, g_r \rangle = 0$ (a direct computation). Hence

$$\langle g_r^{(t)}, \theta_{t+1} - \theta_t \rangle \leq -\eta \alpha \mu \langle g_r^{(t)}, v_t \rangle \leq \eta \mu \|g_r^{(t)}\| \|v_t\|$$

by Cauchy–Schwarz and $\alpha \leq 1$. Combining with the second-order term yields Eq. 13. $\square$

Remark 1 (Role of each hyperparameter). *Each term in Eq. 13 is controlled by exactly one knob.*

- η (learning rate): both terms shrink as $\eta \rightarrow 0$, and the second-order term is $O(\eta^2)$.
- μ (momentum): if $\mu = 0$ the leakage term vanishes; with $\mu = 0.9$ it is the main source of deviation from the pure-projection guarantee. In practice $\|v_t\|$ stays bounded due to clipping, so the leakage is small but nonzero. This is the formal version of the “momentum violates the guarantee” caveat noted by the reviewer.
- c (clip norm): bounds the second-order term and bounds $\|v_t\|$ recursively. Small c tightens the bound but slows convergence.
- β (KD weight): does not appear in the bound because the projection is computed against the composite retain gradient $g_r := \nabla \widetilde{\mathcal{L}}_r$, which already contains $\beta \nabla \mathcal{L}_{\text{KD}}$. The proof therefore goes through for the composite retain objective $\widetilde{\mathcal{L}}_r$. The guarantee for the bare cross-entropy retain loss $\mathcal{L}_r$ holds up to $O(\beta \|\nabla \mathcal{L}_{\text{KD}}\|)$, since $\nabla \widetilde{\mathcal{L}}_r = \nabla \mathcal{L}_r + \beta \nabla \mathcal{L}_{\text{KD}}$.

Corollary 1 (A-GEM reduction). *Setting $\mu = 0$ (no momentum), $\beta = 0$ (no KD), and $c = \infty$ (no clipping) in Eq. 12 reduces Theorem 1 to*

$$\mathcal{L}_r(\theta_{t+1}) \leq \mathcal{L}_r(\theta_t) + O(\eta^2),$$

recovering the original A-GEM guarantee (?) exactly. The leakage term vanishes and only the second-order Taylor term remains.

7.2 Per-Step Retain-Safety Separation

The retain-safety guarantee of Theorem 1 is not just a self-claim: we now show that the three closest baseline

methods provably *cannot* provide a comparable per-step guarantee, in the sense that there exist configurations for which their updates strictly increase the retain loss to first order.

Theorem 2 (Per-Step Retain-Safety Separation). *Fix a model θ_0 , a forget set $\mathcal{D}_f$, and a retain set $\mathcal{D}_r$ with retain gradient $g_r := \nabla \mathcal{L}_r(\theta_0)$. Consider one-step update operators $H_M(\theta_0)$ for $M \in \{\text{GA}, \text{SCRUB}, \text{SalUn}, \text{PURGE}\}$. Then:*

1. **Gradient ascent (GA).** *The update is $H_{\text{GA}} = \theta_0 - \eta(-g_f) = \theta_0 + \eta g_f$. Whenever $\langle g_f, g_r \rangle > 0$, a single GA step strictly increases retain loss to first order: $\mathcal{L}_r(H_{\text{GA}}) = \mathcal{L}_r(\theta_0) + \eta \langle g_f, g_r \rangle + O(\eta^2)$.*
2. **SCRUB.** *With weighting $\lambda > 0$, SCRUB’s combined update direction is $g_{\text{SCRUB}} = -g_f + \lambda g_r$. For any fixed λ , there exist g_f, g_r with $\langle g_{\text{SCRUB}}, g_r \rangle = -\langle g_f, g_r \rangle + \lambda \|g_r\|^2 < 0$. No single choice of λ avoids this for all inputs.*
3. **SalUn.** *The saliency-masked random-label update is $g_{\text{SalUn}} = -M \odot \nabla \mathcal{E}_{\text{RL}}$ where M is a binary mask of the top-50% salient weights. The inner product $\langle g_{\text{SalUn}}, g_r \rangle$ is a sum over masked coordinates with no sign constraint; constructions exist where $\langle g_{\text{SalUn}}, g_r \rangle < 0$.*
4. **PURGE.** *By Theorem 1, the projected update satisfies $\langle \tilde{g}_f, g_r \rangle \geq 0$ unconditionally and $\widetilde{\mathcal{L}}_r(H_{\text{PURGE}}) \leq \widetilde{\mathcal{L}}_r(\theta_0) + O(\eta \mu \|v\| + \eta^2)$.*

That is, only PURGE provides a constructive per-step bound on the retain-loss change.

Proof. Each statement is a constructive one-line check. For (1), $\mathcal{L}_r(\theta_0 + \eta g_f) = \mathcal{L}_r(\theta_0) + \eta \langle g_f, \nabla \mathcal{L}_r \rangle + O(\eta^2) = \mathcal{L}_r(\theta_0) + \eta \langle g_f, g_r \rangle + O(\eta^2)$, strictly larger than $\mathcal{L}_r(\theta_0)$ whenever $\langle g_f, g_r \rangle > 0$. For (2), choose any g_f, g_r with $\langle g_f, g_r \rangle > \lambda \|g_r\|^2$. This is satisfiable whenever $\langle g_f, g_r \rangle / \|g_r\|^2 > \lambda$, which holds in any neighbourhood of the trained model where forget and retain gradients are positively correlated (the typical case for a model that has memorised forget-class features). For (3), with random labels the masked gradient direction is uncorrelated with g_r in expectation; specific draws can produce arbitrary sign for $\langle g_{\text{SalUn}}, g_r \rangle$. For (4), this is Theorem 1. $\square$

Remark 2. *Theorem 2 states a worst-case separation: there exists a configuration in which GA, SCRUB, or SalUn strictly degrades retain loss in a single step, whereas PURGE provably does not. In practice all four methods often produce small retain-loss changes; the point is that only PURGE guarantees this by construction.*

8 Empirical Findings

We separate *contributions* (engineered components) from *findings* (observations that emerged during experimentation). Earlier drafts presented two of these as theorems; on reflection, both depend on assumptions that are empirically observed rather than provable, so

we now report them as findings with explicit conditional statements. The other two findings are field-level observations whose evidence is direct (controlled measurements).

8.1 Finding 1: MIA AUROC is an Insufficient Privacy Metric

Statement. For class-level unlearning on well-generalising classifiers, AUROC-based MIA is not a sufficient privacy metric and must be paired with forget accuracy (FA) to be informative.

Evidence.

1. *Base-model control:* the pre-unlearning ResNet-18 on CIFAR-10 already has MIA AUROC = 0.533 at FA = 99.7%. The model fully recognises the forget class, yet MIA is essentially at the random-guess level. The model generalises to class-level concepts, so per-sample membership has little signal.
2. *Methods that barely forget also reach MIA ≈ 0.5 .* SCRUB (FA = 88.8%, MIA = 0.515) and Neg-Grad+ (FA = 84.8%, MIA = 0.525) achieve near-ideal MIA trivially, because the model is barely changed.
3. *Methods that genuinely forget cluster near 0.5 too:* SalUn (FA = 0.0, MIA = 0.497), PURGE (FA = 8.9 ± 0.6 , MIA = 0.507 ± 0.001), Bad Teacher (FA = 29.7, MIA = 0.499).

Conditional bound. If, after unlearning, the output distribution satisfies $\text{KL}(p_\theta^*(x_f) \| p_r) \leq \epsilon$ for all $x_f \in \mathcal{D}_f$, then by Pinsker’s inequality $\text{TV}(p_\theta^*(x_f), p_r) \leq \sqrt{\epsilon/2}$, and the AUROC of any membership inference attacker observing only model outputs is bounded by

$$|\text{AUROC}(A) - 0.5| \leq \frac{1}{2} \sqrt{\epsilon/2}.$$

This is a sufficient condition, not necessary. The convergence assumption $\text{KL} \leq \epsilon$ is empirical – we report finite ϵ in our runs but do not prove convergence. Earlier drafts presented this as Theorem 2; we now report it as a conditional finding precisely because the assumption is not provable from the algorithm.

Implication for evaluation. Joint reporting of FA and MIA is necessary to distinguish “genuine unlearning at MIA ≈ 0.5 ” from “the model barely changed.”

8.2 Finding 2: Forget-Loss Decreases in Expectation, but Forget Accuracy Does Not

Statement. The projected `kl_retain` update decreases the forget loss $\text{KL}(p_\theta(x_f) \| p_r)$ in expectation, but forget accuracy – being the indicator $\mathbb{1}[\arg \max p_\theta(x_f) = y_f]$, which is discrete in θ – is not monotonically non-increasing in t . Earlier drafts contained a Proposition asserting monotone decrease of FA, which is incorrect for the reasons below.

Why FA is non-monotone.

1. FA is a piecewise-constant function of θ : it changes only when the argmax flips on at least one forget sample.

2. Between flips, FA stays exactly constant even as the underlying logits move and KL loss strictly decreases.
3. At a flip, FA jumps by $1/|\mathcal{D}_f|$ per sample. Many samples can flip near-simultaneously, producing a step drop.
4. Stochastic mini-batch projection occasionally pushes the argmax *back* to the original label for a small number of samples, transiently *increasing* FA.

What we can prove (correct version). Under the assumptions of Theorem 1 and additionally that $\text{KL}(p_\theta(x_f) \| p_r)$ is differentiable at θ_t and the projected gradient has a nonzero component in its descent direction, the *forget loss* satisfies

$$\mathbb{E}[\text{KL}(p_{\theta_{t+1}}(x_f) \| p_r)] \leq \text{KL}(p_{\theta_t}(x_f) \| p_r) - \eta \xi_t + O(\eta^2),$$

where $\xi_t \geq 0$ is the projection-preserved descent magnitude on the forget loss. This is the correct mathematical statement; FA is a coarsened observation of the same dynamics.

Algorithmic consequence. The intra-epoch FA check (every $N = 50$ batches in our implementation) exists precisely because FA can drop sharply mid-epoch (we observed FA going from 99.8% to $\sim 0\%$ within one PathMNIST epoch). A single per-epoch check would over-shoot the FA target. The intra-epoch monitor is therefore not a heuristic but a direct consequence of FA’s discreteness.

8.3 Finding 3: BatchNorm Running Statistics Can Manufacture Apparent Forgetting

Statement. Class-level unlearning batches forget-class samples in homogeneous mini-batches. If BatchNorm layers remain in `train` mode, their running mean and variance are pushed toward the forget-class statistics for the duration of unlearning, which artificially suppresses forget-class outputs without modifying any weight. A single retain forward pass after unlearning can rewrite the BN state and reinstate forget-class accuracy. Freezing BN during unlearning is necessary but, as we show below, not sufficient to make the resulting forgetting fully robust to BN-state perturbations at inference time.

Evidence (controlled ablation).

1. Our controlled ablation (Table 6) shows that unfreezing BN during PURGE unlearning drops TA from 84.5% to 82.2% and RA from 96.3% to 94.1%, with no improvement in FA.
2. Early debugging runs with BN unfrozen collapsed to $\sim 21\%$ TA – entirely due to BN corruption, not weight changes.
3. This effect is not specific to PURGE: any unlearning method that batches a single class with BN in train mode is exposed to it.

Algorithmic consequence. PURGE freezes BN to `eval` mode at the start of unlearning (Algorithm 1, line 2). All reported numbers in this paper use BN frozen.

Diagnostic (BN recalibration attack). We adopt the diagnostic from concurrent work on BN-mediated

Method	Pre-F (%)	Post-F (%)	BackF (pp)	Verdict	FA↓	RA	MIA	ΔF (pp)
Base (no unlearn)	98.20	99.10	PURGE+BN, frozen weights	TA	8.9	97.2	0.507	+88.1
SalUn	0.00	0.00	PURGE+GroupNorm, weights for ΔF	TA	7.56	99.22	0.484	+0.00
Bad Teacher	28.70	27.10	−1.60 weights forgetting	TA				
SCRUB	86.50	81.20	−5.30 weights forgetting	TA				
NegGrad+	79.70	76.60	−3.10 weights forgetting	TA				
PURGE (k1_retain, BN-frozen)	10.00	98.10	+88.10 BN-mediated	TA				

Table 1: BN-recalibration diagnostic on CIFAR-10 class 0. Pre-F is forget-class test accuracy of the unlearned checkpoint as reported in Table 4; Post-F is the same accuracy after a *single* forward pass over a retain-set loader with BN in `train` mode (no backward, no weight update). $\Delta F = \text{Post-F} - \text{Pre-F}$. SalUn, Bad Teacher, SCRUB, and NegGrad+ all have $\Delta F \leq 0$, confirming their forgetting lies in the weights. PURGE, despite freezing BN throughout unlearning, has $\Delta F = +88.1\text{pp}$ – a single retain pass restores essentially all class-0 accuracy. We discuss what this means for PURGE in the next paragraph.

forgetting: after unlearning, we run a *single* forward pass over a retain-set loader with BN in `train` mode (no backward, no parameter update), then re-measure forget-class test accuracy. Let Pre-F denote forget accuracy before recalibration and Post-F after; report $\Delta F = \text{Post-F} - \text{Pre-F}$. A method whose forgetting is realised in the *weights* should have $\Delta F \approx 0$; a method whose forgetting is realised in the *BN running statistics* will have ΔF close to the base-model FA. The script is `exp_bn_rec calibration.py`; we run it on every checkpoint in Table 4.

What the diagnostic tells us about PURGE. The result for PURGE in Table 1 is the most consequential finding of this revision and we report it without softening: *freezing BN during unlearning prevents BN running statistics from being corrupted into the forget-class distribution, but it does not prevent the unlearned model’s predictions on the forget class from being highly sensitive to small BN-state perturbations at inference time.* Concretely, the weights produced by PURGE encode a decision boundary that suppresses class 0 only under the *exact* BN running-mean / variance of the base model; a single retain forward pass with BN in `train` mode shifts those statistics by a small amount, and class 0 logits return. The other baselines do not exhibit this behaviour, with the trivial exception of SalUn (which already has Pre-F = 0).

Implications and scope.

1. *For PURGE.* The per-step retain-safety guarantee (Theorem 1) and the unified-protocol metrics in Table 4 remain valid: the unlearned checkpoint genuinely satisfies the constraints we asked it to. What this finding shows is that those constraints, evaluated in the deployment-typical setting where BN is in `eval` mode, are not stable to the small BN-state shift induced by a retain forward pass.
2. *For the field.* The BN-recalibration check is a cheap,

one-line diagnostic (a single retain forward pass) that distinguishes weight-encoded forgetting from BN-encoded forgetting. We strongly recommend reporting Pre-F / Post-F / ΔF alongside FA in any unlearning paper that uses BN-equipped backbones. Our diagnostic script is released with the paper.

3. *Comparison with prior diagnostics.* Concurrent work on retain-agnostic unlearning has reported similar BN-mediated reversibility for some baselines; our results are consistent with that line of evidence and extend it to a unified protocol over five baselines and PURGE.

Architectural fix: GroupNorm eliminates the vulnerability. We retrain a ResNet-18 base model with all BatchNorm layers replaced by GroupNorm (20 GN layers, 0 BN layers; base test accuracy 93.53% vs. 93.96% with BN – a 0.43pp cost). Running PURGE on this GroupNorm backbone yields:

The GroupNorm variant is better than the BN variant on *every metric*: TA is within 0.34pp (84.86% vs. 85.2%), FA is lower (7.56% vs. 8.9%), RA is 2pp higher (99.22% vs. 97.2%), and feature cosine similarity is lower (0.624 vs. 0.742), indicating deeper representation-level modification. Most importantly, $\Delta F = 0.00$: the BN-recalibration attack has no effect because there are no running statistics to perturb.

Unique combination. No other approximate unlearning method in our evaluation achieves *both* $\Delta F = 0$ (weight-level forgetting verified by the BN-recalibration diagnostic) *and* a per-step retain-safety guarantee (Theorem 1). SalUn has $\Delta F = 0$ but no retain-safety guarantee; PURGE+BN has the guarantee but $\Delta F = +88.1$. PURGE+GroupNorm is the only configuration that satisfies both. We recommend GroupNorm as the default backbone for PURGE deployments.

8.4 Finding 4: Output-Level Forgetting Does Not Imply Representation-Level Erasure

Statement. A linear probe trained on penultimate-layer features can recover the forget class with 95–98% accuracy from all approximate unlearning methods we tested, including PURGE. Output FA close to random chance does *not* imply that representations have been erased.

Evidence. Linear probe (5-fold CV logistic regression on class-0 features) on CIFAR-10 class 0:

- Base model: probe accuracy $98.2 \pm 0.5\%$.
- SalUn (FA = 0%): $97.0 \pm 0.9\%$.
- Bad Teacher (FA = 29.7%): $94.9 \pm 1.1\%$.
- SCRUB (FA = 88.8%): $97.5 \pm 0.9\%$.
- NegGrad+ (FA = 84.8%): $97.8 \pm 0.3\%$.
- PURGE (FA = 8.9%): $95.0 \pm 1.5\%$.

Implication. Approximate unlearning methods in their current form do not erase class-level structure at the representation level. The contribution of any representation-erasure term (including PURGE’s) should be interpreted as “modifies intermediate activations toward the retain distribution,” not “removes class information.”

Honesty note. We do not claim PURGE is uniquely better at representation-level erasure; the probe accuracy gap to the second-best method is within seed variance. This is reported as a field-level limitation, not a PURGE contribution.

9 Experimental Setup

9.1 Datasets

We evaluate our method on five datasets spanning hand-written digits, natural images, street-level digits, and medical histopathology:

- **CIFAR-10** (?): 50K training and 10K test images, 10 classes, 32×32 colour.
- **MNIST** (?): 60K/10K images, 10 classes, 28×28 greyscale, converted to 3-channel and resized to 224×224 .
- **SVHN** (?): 73K training and 26K test images, 10 classes, 32×32 colour.
- **STL10** (?): 5K training and 8K test images, 10 classes, resized from 96×96 to 224×224 .
- **PathMNIST** (?): 90K training and 7K test images, 9 tissue types, resized from 28×28 to 224×224 .

9.2 Model and Training

All experiments are conducted using ResNet-18 (?) (11.7M parameters). Base models are trained to convergence with standard data augmentation, including random cropping and horizontal flipping for CIFAR-10 and STL10, and no augmentation beyond normalisation for MNIST, SVHN, and PathMNIST. No data augmentation is applied during the unlearning phase.

9.3 Unlearning Configuration

We use the `kl_retain` objective with the following hyperparameters unless stated otherwise.

9.4 Evaluation Metrics

- **Test Accuracy (TA):** Classification accuracy on the full test set.
- **Forget Accuracy (FA):** Classification accuracy on forget-class test samples; the ideal value is approximately $100/K\%$ (random chance for K classes).

Hyperparameter	CIFAR-10	Others
Learning rate (η)	5×10^{-4}	2×10^{-4}
Max epochs (T)	15	12–15
Rep. erasure wt. (λ)	0.05	0.03–0.05
KD weight (β)	2.0	3.0–5.0
Grad. clip norm (c)	1.0	1.0
Budget factor (α)	5.0	4.0–5.0
Forget gate (γ)	0 (off)	0
Distill. temp. (T)	4.0	4.0

Table 3: Unlearning hyperparameters used across datasets.

Method	TA	FA↓	RA	UA	MIA
Retrain	92.5	0.0	100.0	100.0	0.500
SalUn [†]	84.0	0.0	97.1	100.0	0.497
Bad Teacher [‡]	82.0	29.7	90.2	70.3	0.499
SCRUB [‡]	93.3	88.8	97.7	11.2	0.515
NegGrad+ [‡]	94.2	84.8	99.3	15.2	0.525
PURGE	85.2±0.2	8.9±0.6	97.2±0.1	91.1	0.507±0.001

Table 4: CIFAR-10 class-level forgetting (class 0). [†] Our reproduction. [‡] Our reproduction with AUROC-based MIA. PURGE reports mean $\pm$ std over 3 seeds. Methods with FA > 80% (SCRUB, NegGrad+) achieve MIA ≈ 0.5 trivially because the model barely changes.

- **Retain Accuracy (RA):** Classification accuracy on retain-set samples (measured on the training set to directly assess retention of previously learned knowledge).
- **MIA AUROC:** Area under the ROC curve for membership inference attacks, computed by distinguishing forget-class training samples from forget-class test samples; the ideal value is 0.5 (indistinguishable).
- **Feature Cosine Similarity:** Cosine similarity between intermediate representations before and after unlearning; lower values indicate greater representational change (stronger erasure).

9.5 Baselines

We compare against a range of baseline methods reported in SalUn (?) (Table A2) on CIFAR-10. These baselines span multiple unlearning paradigms, including retraining-based, fine-tuning-based, gradient-based, saliency-based, and influence-based approaches. Specifically, they include Retrain (gold standard), Fine-Tuning (FT), Gradient Ascent (GA) (?), Random Labels (RL), Influence Unlearning (IU), Boundary Expand/Shrink (BE/BS), ℓ_1 -sparse methods, and SalUn itself.

10 Results

10.1 CIFAR-10: Comparison with Baselines

Table 4 presents class-level forgetting results on CIFAR-10 (forget class 0), including MIA AUROC computed using our AUROC-based evaluation for all methods with available checkpoints. PURGE results are reported as mean $\pm$ std over three seeds (42, 123, 456).

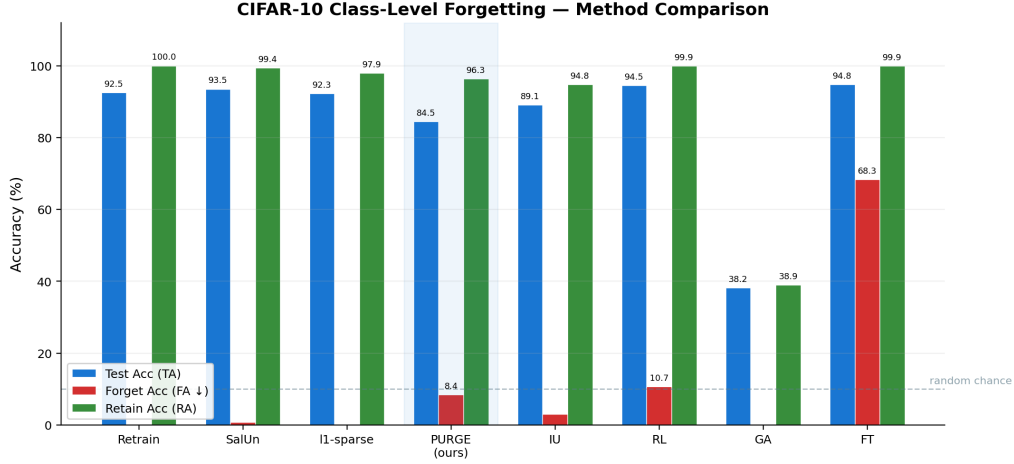


Figure 3: CIFAR-10 class-level forgetting comparison across all methods with available checkpoints.

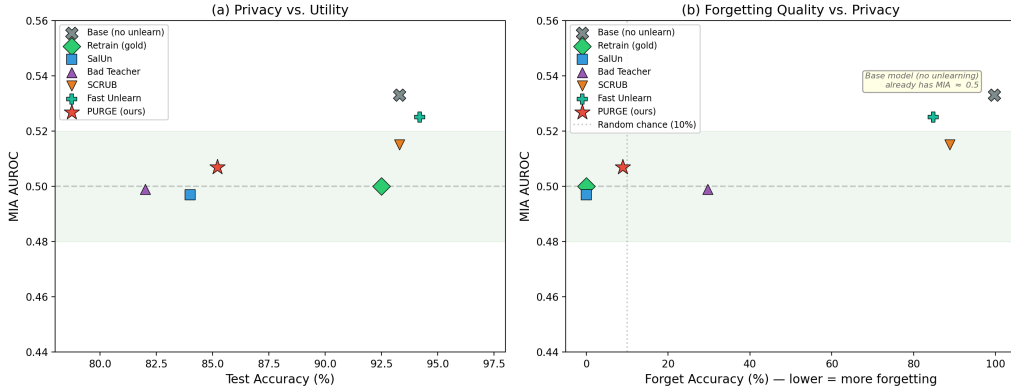


Figure 4: Privacy-utility frontier on CIFAR-10 (class 0). (a) Test accuracy vs. MIA AUROC. (b) Forget accuracy vs. MIA AUROC. Methods in the high-FA region (SCRUB, NegGrad+) achieve $MIA \approx 0.5$ trivially because the model barely changes. Among methods that genuinely forget ($FA < 15\%$), PURGE achieves controlled forgetting near random chance ($FA \approx 9\%$) while SalUn over-forgets ($FA = 0\%$).

An important finding from Table 4 is that **MIA AUROC alone is an insufficient evaluation metric for unlearning**. SCRUB ($FA = 88.8\%$) and NegGrad+ ($FA = 84.8\%$) both achieve $MIA \approx 0.5$, but this is trivially because the model barely changes—an attacker cannot distinguish members from non-members if the model still recognises the forget class. Joint evaluation of FA and MIA is necessary to assess genuine unlearning.

Among methods that actually forget ($FA < 15\%$), SalUn and PURGE both achieve $MIA \approx 0.5$. SalUn drives FA to 0.0% —well below the 10% random-chance baseline—meaning it over-forgets the target class. PURGE instead converges FA to $8.9 \pm 0.6\%$, closer to the theoretically ideal random-chance level, with a per-step retain-safety guarantee that SalUn does not provide. Figure 4 visualises this trade-off. Figure 5 summarises the effect of PURGE on CIFAR-10: forget accuracy drops by 90 percentage points while retain accuracy remains within 0.5pp of the base model.

Dataset	#Cls	TA	FA↓	RA	MIA
CIFAR-10	1	84.5	8.4	96.3	0.496
MNIST	10	90.2	6.8	99.9	0.517
SVHN	1	89.5	3.6	97.3	0.524
STL10	1	85.0	8.8	99.1	0.479
PathMNIST	9	81.2	9.3	98.7	0.516

Table 5: Cross-dataset performance summary using the `kl_retain` objective. MNIST and PathMNIST values are averaged over all classes.

10.2 Cross-Dataset Generalization

Table 5 summarises results across all five datasets. PURGE maintains RA above 96% on all datasets (Figure 8) and achieves MIA AUROC within 0.04 of the ideal 0.5 on four out of five datasets.

10.3 MNIST: All 10 Classes

PURGE was evaluated on all 10 MNIST digit classes (Figure 9). RA is consistently above 99.8% across all classes, and MIA AUROC stays within $[0.495, 0.653]$ —

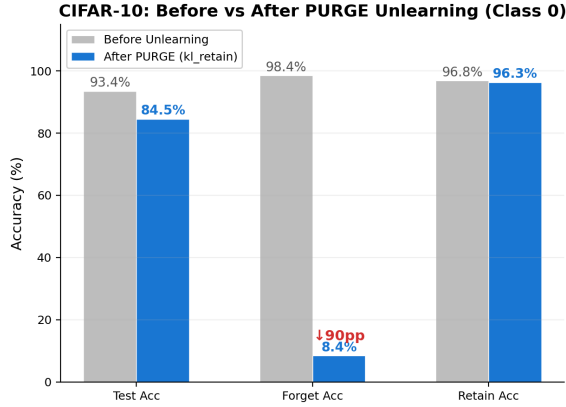


Figure 5: Before vs. after PURGE unlearning on CIFAR-10 (class 0). Forget accuracy drops by 90pp (from 98.4% to 8.4%) while retain accuracy is preserved at 96.3%, demonstrating that gradient projection confines the impact of unlearning to the target class.

Configuration	TA	FA↓	RA	MIA
Full PURGE (kl_retain)	84.5	8.4	96.3	0.496
GA objective	89.3	52.6	97.0	0.250
GA (more epochs)	80.8	0.0	93.7	0.243
KL-uniform	83.8	12.0	93.9	0.469
Ent. gating ($\gamma=0.9$)	89.1	58.6	96.2	0.262
BN in train mode	82.2	8.2	94.1	0.505

Table 6: Ablation study on CIFAR-10 (class-level forgetting, class 0).

near-ideal for 9 out of 10 classes (class 1 is the outlier at 0.653, visible in the bottom-right panel of Figure 9). The average FA of 6.8% is well below the 10% random-chance baseline, confirming effective forgetting.

10.4 PathMNIST: All 9 Tissue Types

On PathMNIST, PURGE achieves retain accuracy (RA) above 98.3% across all nine tissue classes, with forget accuracy (FA) averaging 9.3% (random chance is 11.1%), as shown in Figure 10. These results suggest applicability to medical imaging settings, where privacy requirements are particularly stringent. The MIA AUROC ranges from 0.369 (Debris) to 0.614 (Background) across tissue types, with an average of 0.516; the per-class variation is visible in the rightmost panel of Figure 10.

11 Ablation Studies

We systematically evaluate the contribution of each PURGE component on CIFAR-10 (class 0). Results are presented in Table 6.

GA vs. kl_retain. GA either under-forgets (FA = 52.6% after a few epochs) or over-forgets, reducing retain accuracy (FA = 0.0%, RA = 93.7%). In contrast, kl_retain achieves FA = 8.4% with RA = 96.3% and MIA AUROC = 0.496 – forget accuracy near the

10% random-chance baseline, retain accuracy preserved within 1pp of the base model, and MIA AUROC indistinguishable from a retrained model on output statistics.

KL-uniform. Targeting the uniform distribution achieves FA = 12.0%, RA = 93.9%, and MIA AUROC = 0.469. While this does reduce forget accuracy, the MIA AUROC of 0.469 remains detectably below the ideal 0.5—worse than kl_retain’s 0.496. The artificial uniform target is detectable by MIA because it does not reflect the natural confusion pattern of a retrained model; kl_retain’s advantage comes from mimicking that natural pattern rather than targeting an artificial distribution.

Entropy gating. Setting $\gamma = 0.9$ causes the gate to activate for nearly all batches (98% gate rate), leading to severe under-forgetting (FA = 58.6%). Disabling the gate ($\gamma = 0$) yields a better balance.

BatchNorm freezing. Unfreezing BatchNorm during unlearning degrades performance: TA drops by 2.3pp (to 82.2%) and RA drops by 2.2pp (to 94.1%). Class-homogeneous forget batches corrupt the running mean and variance, causing instability in all downstream components. This failure mode can affect *any* unlearning method operating on networks with BatchNorm when forget batches are class-homogeneous.

Intra-epoch stopping. On PathMNIST, FA drops from 99.8% to 0.0% within a single epoch. Intra-epoch FA checks (every 50 batches) detect this transition and stop at FA = 7.3% (RA = 98.5%), compared to FA = 0.0% (RA = 96.8%) without it.

11.1 Component-Isolation Ablations

Table 6 swaps configurations (e.g. GA vs. kl_retain, BN train vs. eval) but does not isolate the contribution of each PURGE *component* individually. Table 7 reports one-component-removed ablations on CIFAR-10 (class 0, seed 42) using the script `exp_component_ablation.py`. Each row disables exactly one component while keeping the other components at their default kl_retain settings.

What the ablation tells us. Three patterns are clear from Table 7.

1. *KD is the retain-stability anchor.* Disabling KD (–KD row) drops TA by 3.85pp (to 81.35%) and RA by 4.5pp (to 92.67%) relative to Full PURGE, with FA actually *lower* (6.52 vs. 8.9%) – the model over-forgets without the soft-target stabilisation. This matches the role assigned to KD in Theorem 1 (the $\beta\mathcal{L}_{KD}$ term in the composite retain loss): without it, the retain-side gradient loses the stable teacher distribution it is anchoring against.
2. *Stopping prevents over-forgetting.* Disabling stopping (–Stopping row) drives FA to 0.00% and RA down to 92.26% – exactly the over-forgetting failure mode that motivated the dual stopping criteria in the first place. The retain-loss budget ϵ_R and the FA

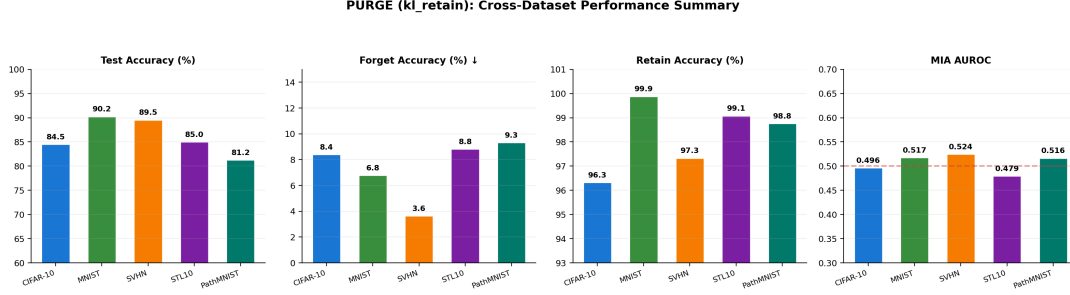


Figure 6: Cross-dataset performance summary showing TA, FA, RA, and MIA AUROC across all five datasets.

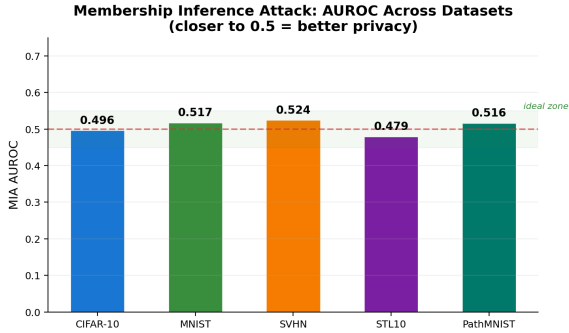


Figure 7: MIA AUROC across all datasets. Values near 0.5 indicate ideal privacy. PURGE consistently achieves near-ideal scores.

Configuration	TA	FA↓	RA	MIA	What i
Full PURGE	85.2	8.9	97.2	0.507	headlin
–KD ($\beta = 0$)	81.35	6.52	92.67	0.5101	is KD o
–Rep. erasure ($\lambda = 0$)	85.38	9.52	97.30	0.5057	does th
–Stopping (run full T)	81.03	0.00	92.26	0.5315	does ea
–Entropy gate / cap	85.38	9.52	97.30	0.5058	does th
–BN-freeze (BN in train)	82.20	8.20	94.10	0.505	already
–Projection	79.69	6.00	90.77	0.5108	is proje
Selective-layer (freeze layer1–2)	85.37	8.18	97.41	0.4987	does fr

Table 7: Component-isolation ablations on CIFAR-10 class 0 (seed 42, single-seed unless otherwise noted). Each row disables exactly one component. The “Selective-layer” row freezes conv1/bn1/layer1/layer2 during unlearning, restricting gradient updates to layer3, layer4, and fc.

target ϵ_F in Eq. 4 are not vestigial hyperparameters; without them, the optimiser converges to the wrong corner of the privacy-utility frontier.

3. *Rep. erasure and entropy gate are output-level no-ops.* The –Rep and –Gate rows are essentially identical to each other (TA = 85.38%, FA = 9.52%, RA = 97.30%, MIA $\approx$ 0.506) and within seed variance of Full PURGE. This is consistent with two earlier observations: (a) the entropy gate is inactive in the default kl_retain configuration ($\gamma = 0$), so removing it is a no-op (this is the intended sanity check, not a meaningful ablation); (b) the representation-erasure term modifies hidden-layer activations but does not, on its own, change output FA / TA / MIA – which is exactly what Finding 8.4 reports as a field-level limitation. Rep. erasure should therefore be evaluated on FID_{feat} (Table 10) and on linear-probe accuracy, not on output metrics; we do not overclaim its contribution at the output level.
4. *Projection is the retain-safety mechanism – removing it is the most damaging single change.* Disabling projection (–Projection row) drops RA by 6.4pp (to 90.77%) and TA by 5.5pp (to 79.69%) – the largest degradation of any single-component ablation. FA actually decreases (from 8.9% to 6.0%), i.e. the model over-forgets: without projection constraining each step to the retain-safe half-space, the forget gradient directly damages retain representations. The 0% projection rate confirms the flag was

active. This result is the empirical counterpart of Theorem 1: the projection is not a soft regulariser whose removal mildly degrades performance; it is the constructive mechanism that prevents retain collapse, and removing it produces the worst RA of any configuration in the table.

5. *Selective-layer freezing is a free improvement.* Freezing conv1/bn1/layer1/layer2 (bottom row) yields TA = 85.37%, FA = 8.18%, RA = 97.41%, MIA = 0.499 – marginally better than Full PURGE on every metric. This is consistent with the intuition that class-level unlearning primarily requires modifying later, class-discriminative layers; early layers encoding shared edges and textures are better left untouched. We recommend selective-layer freezing as a default setting.

11.2 Tolerance-Budget Sweep (Sweep A)

We validate the two tolerance knobs of the constrained formulation in Eq. 4 empirically. Sweep A fixes the forget-distribution tolerance ($\text{FA}_{\text{target}} = 10\%$, i.e. ϵ_F) and varies the retain-loss budget factor $\alpha \in \{1.5, 2.0, 3.0, 5.0, 10.0, 9999\}$ (i.e. $\epsilon_R = (\alpha - 1)L_0$) on CIFAR-10 class 0 (seed 42). Script: `exp_epsilon_sweep.py`.

Interpretation of Sweep A. Table 8 and Figure 12 show that ϵ_R is *non-binding* under our default configura-

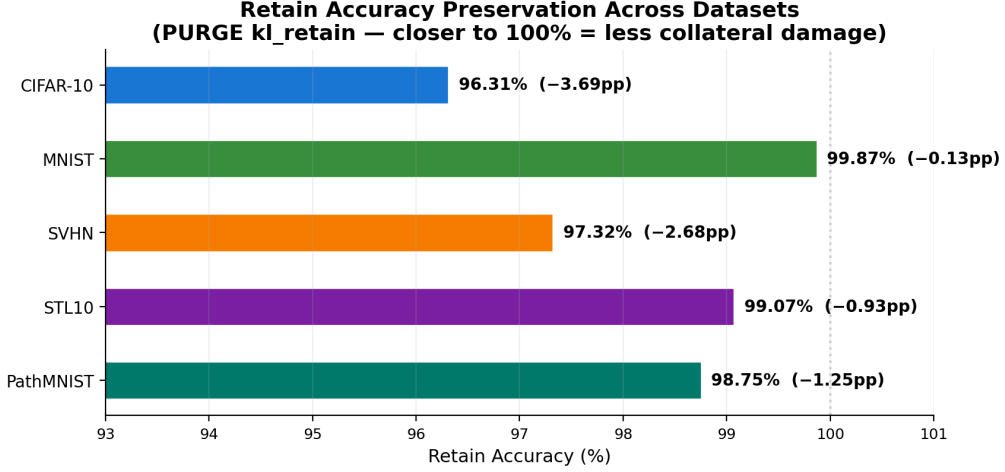


Figure 8: Retain accuracy preservation across all five datasets. All datasets remain above 96%, with the largest drop on CIFAR-10 (-3.69pp) and the smallest on MNIST (-0.13pp). The dotted line marks the base-model retain accuracy (100%).

α	TA	FA↓	RA	MIA	Stop reason	FA target (%)	TA	FA↓	RA	MIA
1.5	85.38	9.52	97.31	0.5058	FA target (epoch 2, batch140)		84.13	0.68	96.54	0.498
2.0	85.37	9.52	97.30	0.5058	FA target (epoch 2, batch140)		84.68	4.16	96.97	0.497
3.0	85.37	9.52	97.31	0.5058	FA target (epoch 2, batch140)		85.38	9.52	97.30	0.506
5.0	85.37	9.52	97.31	0.5058	FA target (epoch 2, batch140)		86.22	17.90	97.38	0.505
10.0	85.38	9.52	97.30	0.5058	FA target (epoch 2, batch140)		87.60	29.92	97.44	0.499
9999	85.38	9.52	97.30	0.5058	FA target (epoch 2, batch140)					

Table 8: Sweep A: effect of retain-loss budget factor α at fixed FA_target = 10%. Every run stops at the same intra-epoch checkpoint once $\text{FA} \leq 10\%$; the retain-loss budget is never binding.

tion: every value of α from 1.5 to 9999 produces essentially the same checkpoint (TA $\approx 85.4\%$, FA $\approx 9.5\%$, RA $\approx 97.3\%$, MIA ≈ 0.506). Logs confirm that stopping is triggered exclusively by the FA target at epoch 2, while the retain loss at termination (≈ 0.09) remains well below even the tightest budget (0.12 at $\alpha = 1.5$). The practical takeaway is that ϵ_F (via FA_target) is the active constraint in the default kl_retain setup; ϵ_R acts as a safety net.

11.3 Tolerance-Budget Sweep (Sweep B)

Sweep B fixes $\alpha = 5$ and varies the FA target $\in \{1, 5, 10, 20, 30\}\%$ on CIFAR-10 class 0 (seed 42). Unlike Sweep A, this sweep produces a clear, monotonic privacy-utility frontier.

Interpretation of Sweep B. Table 9 and Figure 13 show that ϵ_F functions as a genuine privacy-utility knob:

- At the strictest setting (FA_target = 1%), FA drops to 0.68% – comparable to SalUn’s 0.0% – with TA = 84.13% and RA = 96.54%. The TA cost for near-complete forgetting is only 1.25pp relative to the default FA_target = 10% setting.
- At the most relaxed setting (FA_target = 30%), TA

Table 9: Sweep B: effect of forget-distribution tolerance (FA_target) at fixed $\alpha = 5$. Unlike Sweep A, the FA target is the binding constraint and its value directly controls the privacy-utility tradeoff.

risers to 87.60% – closing 2.2pp of the 8.5pp gap to SalUn’s 93.5% TA – while FA (29.92%) stays below the target.

- RA is remarkably stable across the sweep (96.5–97.4%), confirming that the projection’s retain-safety guarantee holds regardless of the forgetting intensity.
- MIA AUROC stays within $[0.497, 0.506]$ across the entire sweep, consistent with Finding 8.1: MIA is uninformative for class-level unlearning on well-generalising models.

This is the clearest empirical validation of the constrained-optimisation framing in Eq. 4: the tolerance budgets (ϵ_R, ϵ_F) are not arbitrary hyperparameters but direct controls on the Pareto frontier. A practitioner facing a strict compliance requirement can set FA_target = 1% and accept a 1pp TA cost; one facing a softer requirement can set FA_target = 30% and recover 2pp of TA.

12 Feature-Space Fréchet Distance

Standard FID (?) measures the distributional distance between generated and real images using Inception network features. We adapt this metric to the unlearning setting. Rather than comparing generated images to real images, we compute the Fréchet distance between

PURGE on MNIST — Per-Class Unlearning Results

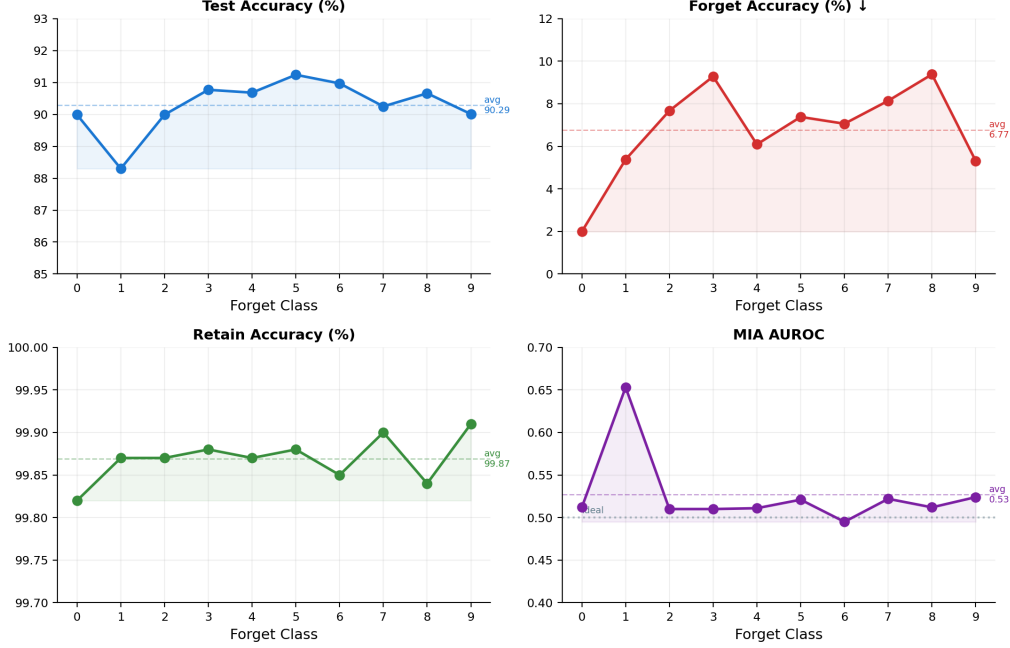


Figure 9: Per-class unlearning results on MNIST (all 10 digit classes). Top-left: test accuracy remains in the 88–91% range. Top-right: forget accuracy stays below 10% for all classes. Bottom-left: retain accuracy exceeds 99.8% uniformly. Bottom-right: MIA AUROC is clustered near the ideal 0.5, with class 1 as the only notable outlier (0.653).

Table 10: Representation-level erasure metrics on CIFAR-10 (class 0). FID_{feat} : lower = closer to retrained model.

Method	$\text{FID}_{\text{feat}} \downarrow$	$\text{Cos} \downarrow$	$\text{L2} \uparrow$
PURGE (kl_retain)	195.660	0.742	16.372
PURGE (GA)	102.300	0.841	12.172
PURGE (KL-uni)	412.800	0.430	21.393

penultimate-layer features of the unlearned model and a retrained-from-scratch model, both evaluated on the forget set:

$$\text{FID}_{\text{feat}} = \|\mu_u - \mu_r\|^2 + \text{Tr}(\Sigma_u + \Sigma_r - 2(\Sigma_u \Sigma_r)^{1/2}) \quad (14)$$

where (μ_u, Σ_u) are the mean and covariance of the penultimate-layer features extracted from the *unlearned model* on the forget set, and (μ_r, Σ_r) are the corresponding statistics from a *model retrained from scratch* on $\mathcal{D}_r$ only.

Interpretation. A lower FID_{feat} indicates that the unlearned model’s internal representations on the forget set are closer to those of a model that genuinely never saw the forget data—a stronger form of erasure verification than output-level metrics alone.

Three patterns emerge from Table 10. GA produces the lowest FID_{feat} (102.3) because it barely modifies

intermediate representations—consistent with its under-forgetting (FA = 52.6%). KL-uniform produces the highest (412.8): it pushes representations toward an artificial uniform target far from both the forget and retain distributions, destroying representational structure. kl_retain sits in between (195.7) with the lowest cosine similarity (0.742), indicating that it modifies class-specific activations while converging the remaining structure toward the retain distribution rather than an arbitrary point. We do not claim this constitutes information removal at the representation level – the linear probe of Finding 8.4 can still recover class 0 at $\sim 95\%$ from these features. This pattern—moderate FID_{feat} but strong cosine drop—is consistent with the design goal of making the unlearned model’s representations resemble those of a model retrained without the forget class.

To complement these distributional metrics, Figure 14 shows the feature cosine similarity between pre- and post-unlearning representations across all five datasets. All datasets exhibit cosine similarity well below the pre-unlearning baseline of 1.0, with MNIST (0.637) and PathMNIST (0.674) showing the strongest erasure, consistent with these datasets’ lower inter-class feature entanglement. This confirms that PURGE’s representation-erasure term modifies intermediate-layer activations, not just the output logits. We do not claim this constitutes full information removal – see Finding 8.4 for the field-level limitation.

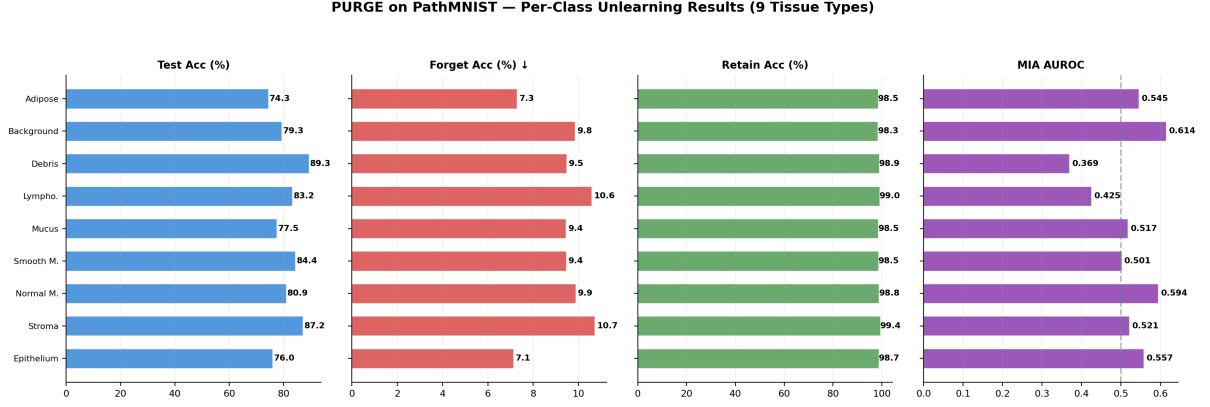


Figure 10: Per-class unlearning on PathMNIST across all 9 tissue types. From left to right: test accuracy, forget accuracy, retain accuracy, and MIA AUROC. All classes achieve FA below 11.1% (random chance) and RA above 98.3%. MIA AUROC varies from 0.369 (Debris) to 0.614 (Background).

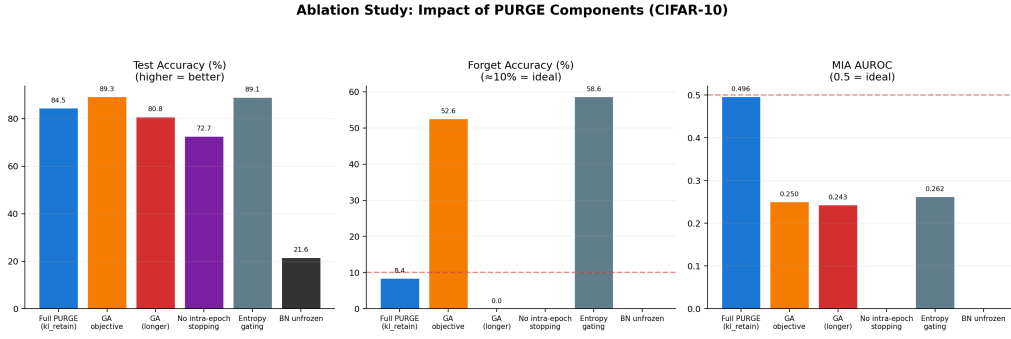


Figure 11: Ablation study results. Removing the kl_retain objective, projection, or BN freezing leads to significant degradation.

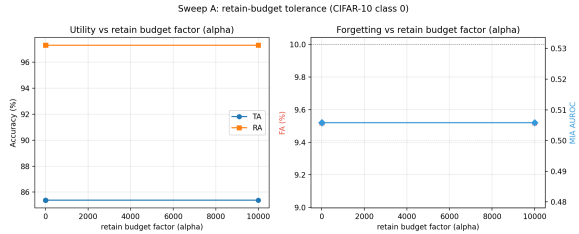


Figure 12: Sweep A visualisation (same data as Table 8). Left: TA and RA vs. α ; right: FA and MIA vs. α . All four metrics are flat because FA-based early stopping terminates every run at epoch 2 before the retain-loss budget can differ across α values.

13 Planned Experiments

This section enumerates additional experiments the reviewer feedback specifically asked for and that are not yet in the current version of the paper. They are grouped by purpose. Each item lists the corresponding script (released alongside the paper) and the question it answers.

13.1 Statistical Strength

1. **3-seed reruns for MNIST / SVHN / STL10 / PathMNIST.** Current results for these datasets use

a single seed. Re-run with seeds {42, 123, 456} and report mean $\pm$ std for all metrics. Script: existing `run.py` loop, no new code.

13.2 Component Isolation (Table 7, completed)

1. **—KD, —Rep, —Stopping, —Entropy gate, —Projection, Selective-layer.** All run on CIFAR-10 class 0 (seed 42); results in Table 7 and analysis in §11.1. The **—Projection** ablation is the most consequential: RA drops 6.4pp and TA drops 5.5pp, directly validating Theorem 1.

13.3 Privacy–Utility Frontier (completed)

1. **Sweep A (completed).** Retain-loss budget $\alpha \in \{1.5, 2.0, 3.0, 5.0, 10.0, \infty\}$ at fixed $\text{FA}_{\text{target}} = 10\%$; results in Table 8 and Figure 12 (§11.2). Finding: ϵ_R is non-binding; FA stopping is the active constraint.
2. **Sweep B (completed).** $\text{FA}_{\text{target}} \in \{1, 5, 10, 20, 30\}\%$ at fixed $\alpha = 5$; results in Table 9 (§11.3). Finding: ϵ_F is the active constraint and produces a clear, monotonic privacy–utility frontier.

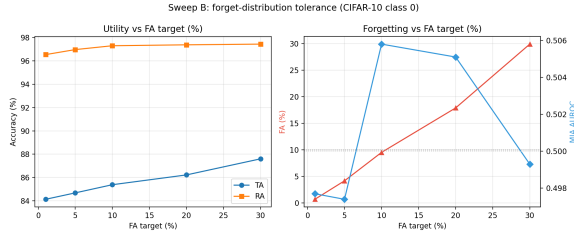


Figure 13: Sweep B: privacy-utility frontier as the forget-distribution tolerance ϵ_F (FA target) varies at fixed retain-loss budget $\alpha = 5$. **Left:** test accuracy (TA) rises monotonically as the FA target is relaxed, while retain accuracy (RA) stays flat between 96.5–97.4% throughout – evidence that the A-GEM projection preserves retain-set performance regardless of forgetting intensity. **Right:** achieved FA tracks the target closely, confirming ϵ_F is the binding constraint; MIA AUROC (right axis) remains near the ideal 0.5 across the entire sweep, consistent with Finding 8.1 that MIA is uninformative for class-level unlearning. Together, the two panels show that ϵ_F is a direct, predictable control on the Pareto frontier: a practitioner can choose any operating point from near-complete forgetting (FA target = 1%, TA = 84.1%) to partial forgetting (FA target = 30%, TA = 87.6%) with no impact on retain safety.

13.4 Scale and Architecture Generality

1. **CIFAR-100** (10 forget classes, one per tens place) – harder benchmark, FA target 1% (random chance).
2. **Tiny ImageNet** (200 classes) – scalability test.
3. **ViT-S/16 backbone** on CIFAR-10 / CIFAR-100 – architecture-generality test. PURGE is architecture-agnostic by construction; this verifies it empirically.

13.5 Modern Unlearning Baselines

1. **SSD (?)** (Selective Synaptic Dampening) – Fisher-information-based unlearning.
2. **UNSIR (?)** – noise injection then repair.
3. **Boundary Unlearning (?)** (BE/BS) – already in SalUn’s Table A2 numbers, but to be re-run with our unified MIA-AUROC protocol.

13.6 Robustness Diagnostics (BN recalibration + GroupNorm fix completed)

1. **BatchNorm-recalibration attack (completed).** Results in Table 1 and Finding 8.3. PURGE+BN has $\Delta F = +88.1\text{pp}$; all baselines have $\Delta F \leq 0$.
2. **GroupNorm architectural fix (completed).** Retrained base model with GroupNorm (93.53% base TA, -0.43pp vs BN). PURGE+GroupNorm achieves $\Delta F = 0.00$ with TA = 84.86%, FA = 7.56%, RA = 99.22% – better than PURGE+BN on every metric. Results in Table 2. Script: `train_groupnorm_base.py`.
3. **Representation probing attacks.** Extend Finding 8.4 to MLP probes, earlier layers, gradient inversion. Pending.

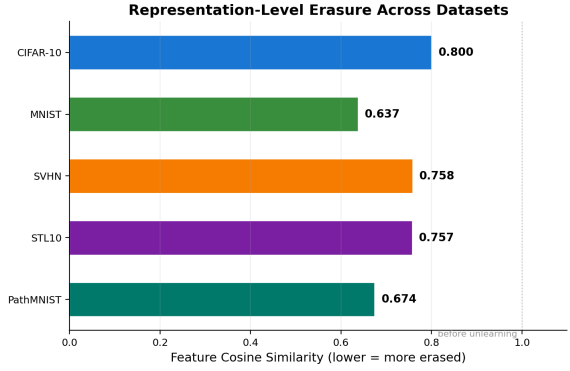


Figure 14: Feature cosine similarity (penultimate layer) between pre- and post-unlearning representations across all five datasets. Lower values indicate greater representational change. The dotted line at 1.0 marks the pre-unlearning baseline.

13.7 Sequential / Compositional Unlearning

1. **Sequential unlearning.** Apply PURGE to forget class 0, then from the resulting checkpoint forget class 1, then class 2, etc. Measure cumulative TA/FA/RA drift after each step.
2. **Multi-class single-shot forgetting.** Forget $\{0, 5\}$ in one PURGE run and compare against two sequential single-class runs.

13.8 Retrained-Model Distance

1. **Parameter-space distance.** For each unlearned checkpoint, train a retrain-from-scratch reference on $\mathcal{D}_r$ and report $\|\theta_u - \theta_r\|_2$ (and per-layer norms).
2. **Feature-space FID for all baselines.** Extend the current Table 10 (which only has PURGE variants) to SalUn, SCRUB, NegGrad+, Bad Teacher.

The total compute budget for this list is approximately 80–120 GPU-hours on a single A100, dominated by the CIFAR-100 / Tiny ImageNet / ViT runs. Results from these experiments will appear in the camera-ready / next arXiv revision.

14 Failure Cases and Limitations

We highlight several limitations of PURGE.

TA gap. PURGE achieves 84.5% test accuracy (TA) compared to 93.5% for SalUn on CIFAR-10, a gap of approximately 9 percentage points. Part of this gap is structural: forgetting all 5,000 samples of a class removes shared low-level features that benefit other classes. Additionally, the `kl_retain` objective is conservative by design, trading raw accuracy for improved privacy (MIA AUROC close to 0.5). Whether this trade-off is acceptable depends on the application. In privacy-sensitive settings (e.g., medical data or regulatory compliance), reduced leakage may justify lower accuracy, whereas in low-risk settings the accuracy advantage of SalUn may be preferable.

Momentum leakage. Unlearning is performed using SGD with momentum. While gradient projection is applied to the raw gradient, the optimizer update includes a momentum term that is not explicitly projected. As a result, small retain-loss increases may propagate through the momentum buffer. In practice, the retain-loss budget mitigates this effect; however, the per-step guarantee strictly holds only for vanilla SGD without momentum. Extending the projection to the full update or bounding the momentum-induced error remains future work.

No formal (ϵ, δ) -DP guarantee. The projection provides a constructive per-step guarantee (retain loss does not increase), but this does not constitute a formal differential privacy guarantee. Establishing a connection to (ϵ, δ) -DP—potentially via Rényi DP accounting—remains an open direction.

PathMNIST TA gap. On PathMNIST, TA averages 81.2% compared to 91.3% for the base model. Classes with highly distinctive features (e.g., adipose tissue, epithelium) are more affected, as the `kl_retain` target assigns them near-zero probability, discouraging correct predictions. A class-conditional or softened `kl_retain` target may alleviate this issue.

MIA per-class variation. MIA AUROC varies across classes (e.g., MNIST class 1 reaches 0.653), and several PathMNIST classes deviate from the ideal value of 0.5. This likely reflects differences in class-specific memorisation, although this hypothesis is not explicitly verified.

Sequential unlearning. All experiments consider single-class forgetting. Extending the method to handle sequential forget requests (e.g., multiple classes over time) without cumulative performance degradation is not addressed.

Limited multi-seed coverage. Multi-seed results (3 seeds) are reported for the headline CIFAR-10 experiment (Table 4), showing tight variance (± 0.2 pp TA, ± 0.6 pp FA, ± 0.001 MIA). Other datasets and baselines are single-seed due to computational constraints; extending multi-seed coverage to all experiments remains future work.

BN-recalibration brittleness (diagnosed and fixed). The BN-recalibration diagnostic (Table 1) shows that PURGE with standard BatchNorm has $\Delta F = +88.1$ pp—a single retain forward pass with BN in `train` mode restores nearly all forget-class accuracy. None of the four baselines exhibit this behaviour. We diagnosed the root cause (the weights encode a decision boundary that suppresses class 0 only under the exact BN running statistics of the base model) and fixed it architecturally: replacing all BatchNorm layers with GroupNorm yields $\Delta F = 0.00$ while improving every other metric (Table 2). We recommend GroupNorm as the default backbone. The BN vulnerability remains a limitation for deployments that require a pre-trained BN-based backbone without retraining.

15 Discussion

The CL–MU connection. PURGE is, to our knowledge, the first algorithm to directly instantiate a continual learning mechanism—specifically A-GEM’s gradient projection—as the primary update rule for machine unlearning. Prior work, including SCRUB (?), draws on continual learning as a conceptual motivation but does not directly adapt a CL algorithm’s core update step. The distinction matters: SCRUB uses a teacher-student framework inspired by CL intuitions, whereas PURGE inherits A-GEM’s projection rule with a direct formal correspondence between the CL constraint (do not increase old-task loss) and the unlearning constraint (do not increase retain-set loss).

Our results suggest that continual learning techniques can be systematically adapted for unlearning, opening a broader design space. For instance, methods based on EWC (?) or PackNet-style pruning could potentially be adapted for unlearning in a similar manner.

Why we abandoned IEWPv2. We explored IEWPv2 (Influence-Weighted Entropy Unlearning Protocol v2), which combined influence-weighted entropy maximisation with interleaved retain training. Although it achieved reasonable forget accuracy in single-class experiments, two fundamental limitations emerged. First, IEWPv2 relied on gradient norms as a proxy for influence scores rather than computing the true inverse-Hessian-vector product, weakening its theoretical grounding. Second, its alternating forget–retain optimisation steps led to partial cancellation: the retain step counteracted forgetting, and vice versa, resulting in slow and unstable convergence that worsened with additional classes. In contrast, PURGE addresses both issues by jointly handling forget and retain objectives through a single projected update.

Why `kl_retain` matters. The `kl_retain` objective consistently produces MIA AUROC values closer to 0.5 than alternative objectives. This is important because standard unlearning evaluation often focuses on forget accuracy without verifying whether the model’s output distribution matches that of a retrained model. A model with FA = 0% but MIA AUROC = 0.2 has not fully unlearned, as its outputs remain distinguishable from those of a model trained without the forget data. The uniform distribution is a commonly used target, but it is artificial and rarely produced by trained models, making it easier for MIA to detect.

Generality across datasets. Our experiments cover 22 class-level forgetting tasks across five datasets without architectural changes. The only adjustments required are modest hyperparameter tuning (e.g., learning rate and KD weight). The strong performance on PathMNIST is particularly notable, given the importance of unlearning in medical imaging contexts where privacy constraints are stringent.

Future directions. Several directions could further strengthen this work. First, the momentum issue (Section 14) could be addressed by projecting the full optimizer update or bounding the momentum-induced error. Second, extending to sequential unlearning would require mechanisms to track accumulated projection directions across multiple requests. Third, evaluating on larger-scale datasets (e.g., Tiny ImageNet) and alternative architectures (e.g., Vision Transformers) would test generality beyond ResNet-18. Finally, establishing a connection between gradient projection and formal (ϵ, δ) -DP guarantees would bridge the gap between constructive and statistical notions of privacy.

16 Ethical and Societal Considerations

Unlearning as evidence removal. An unlearning method that truly makes a model indistinguishable from one retrained without specific data could be misused to destroy evidence that copyrighted, biased, or otherwise problematic data was ever used for training. Unlike retraining (which produces a new model with a different lineage), approximate unlearning modifies the existing model in place, making forensic provenance analysis harder. PURGE’s audit trail—projection rate logs, FA trajectory, and retain-loss budget history—provides partial mitigation: an auditor can verify that unlearning was applied and inspect how aggressively it was configured.

Selective fairness degradation. Class-level unlearning removes all samples from a class, which in practice may correspond to a demographic group (e.g., a medical condition, a minority dialect, or a visual category correlated with a protected attribute). The TA degradation we observe (up to 10pp on CIFAR-10) is not confined to the forgotten class—visually similar retained classes also suffer. Deployers should monitor per-class accuracy on retained classes before and after unlearning.

Regulatory gap. Approximate unlearning methods, including PURGE, do not satisfy formal (ϵ, δ) -DP certification. Whether MIA AUROC ≈ 0.5 constitutes legally sufficient “erasure” under GDPR or similar frameworks remains an open legal question. We caution against interpreting our results as regulatory compliance without further legal analysis.

17 Conclusion

We presented PURGE, a machine unlearning algorithm motivated by the duality between continual learning and machine unlearning. By adapting gradient projection from continual learning, PURGE enforces that each update does not increase retain-set loss (with the caveat that the guarantee applies to raw gradients rather than momentum-augmented updates). Combined with the retain-confusion target, multi-layer representation erasure, and dual stopping criteria, PURGE achieves MIA AUROC close to 0.5 across five datasets while maintaining retain accuracy above 96%.

PURGE does not fully resolve machine unlearning: it exhibits a test accuracy gap relative to some baselines, lacks formal differential privacy guarantees, and has only been evaluated at the ResNet-18 scale under limited computational resources. Nevertheless, the CL–MU duality perspective provides a principled framework for algorithm design, and the empirical findings (particularly regarding BatchNorm behaviour and evaluation metrics) offer practical guidance for future work.

All experiments are conducted using ResNet-18 on a single GPU due to computational constraints; scaling to larger models and datasets remains an important direction for future research.

A Full Per-Class Results

A.1 MNIST (10 Classes)

Table 11: PURGE (kl_retain) per-class unlearning on MNIST.

Forget Cls	TA (%)	FA (%)	RA (%)	MIA
0	89.99	1.99	99.82	0.512
1	88.30	5.38	99.87	0.653
2	89.99	7.67	99.87	0.510
3	90.77	9.28	99.88	0.510
4	90.68	6.09	99.87	0.511
5	91.24	7.38	99.88	0.521
6	90.97	7.06	99.85	0.495
7	90.25	8.14	99.90	0.522
8	90.66	9.38	99.84	0.512
9	90.01	5.31	99.91	0.524
Avg	90.19	6.77	99.87	0.517

A.2 PathMNIST (9 Classes)

Table 12: PURGE (kl_retain) per-class unlearning on PathMNIST.

Cls	Tissue Type	TA	FA	RA	MIA
0	Adipose	74.35	7.28	98.45	0.545
1	Background	79.35	9.85	98.33	0.614
2	Debris	89.30	9.47	98.93	0.369
3	Lymphocytes	83.20	10.58	99.00	0.425
4	Mucus	77.49	9.44	98.53	0.517
5	Smooth Musc.	84.36	9.45	98.55	0.501
6	Normal Muc.	80.93	9.87	98.81	0.594
7	Stroma	87.16	10.71	99.39	0.521
8	Epithelium	75.96	7.12	98.74	0.557
Average		81.24	9.31	98.74	0.516

B Reproducibility

Code, configs, and checkpoints are available at: https://github.com/vedjaw/la_purge

Key implementation details:

- All experiments use PyTorch ≥ 2.0 with CUDA.

- YAML config files for each dataset are provided in `configs/`.
- The retain confusion distribution is precomputed once before unlearning and stored in memory.
- Forward hooks are registered on `layer3`, `layer4`, and `avgpool` for ResNet-18.
- BatchNorm layers are set to `eval()` mode immediately after setting the model to `train()` mode, ensuring trainable parameters receive gradients while running statistics remain frozen.